telcoin network

# Telcoin Network node guide for mobile network operators

Deploy, sync, stake and operate a node

Version 1.6 · November 2026

devs@telcoin.org
https://github.com/Telcoin-Association/tn-node-deployment

telcoin association

# Contents


1 **Introduction** ..... 4

1.1 What this guide covers ..... 4

1.2 Contact ..... 5

1.3 Vocabulary ..... 5

1.4 Roles are decided on-chain ..... 5

2 **Before you start** ..... 7

2.1 Hardware per role ..... 7

2.2 Operating system ..... 8

2.3 Ports ..... 9

2.4 Have these ready ..... 9

2.5 Networks and endpoints ..... 10

3 **Install** ..... 12

3.1 Get the scripts ..... 12

3.2 How the scripts and releases are checked ..... 12

3.3 Run setup ..... 13

3.4 Install methods ..... 15

3.5 How setup stores the passphrase ..... 15

3.6 Where things land ..... 17

3.7 Back up the keys now ..... 19

3.8 Firewall ..... 20

3.9 Check the host hardening ..... 21

3.10 Host baseline (optional) ..... 22

3.11 Optional node flags ..... 23

3.12 Logs ..... 24

4 **Confirm the node is syncing, then synced** ..... 25

4.1 The epoch section ..... 25

4.2 Checking by hand ..... 26

4.3 How long it takes ..... 26

5 **Day-2 operations** ..... 27

5.1 Keep the scripts current ..... 27

5.2 Update the node ..... 28

5.3 One-way updates ..... 30

5.4 Restarts and the epoch boundary ..... 31

5.5 Change the configuration ..... 34

5.6 Service commands ..... 36

5.7 Node Manager UI ..... 36

5.8 Testnet add-ons ... 38
5.9 Key lifecycle ... 40
5.10 Remove a node ... 41

**6 Public RPC (https and wss) ... 42**
6.1 DNS first ... 42
6.2 New install ... 42
6.3 Existing node ... 43
6.4 Verify ... 44
6.5 Turn it off ... 44

**7 Validators: stake and activate ... 45**
7.1 Prerequisites ... 45
7.2 Check the node with prepare-stake.sh ... 45
7.3 Send the stake ... 47
7.4 Activate ... 47
7.5 If prepare-stake.sh predicts a revert ... 48
7.6 Change the execution address before you stake ... 49
7.7 Watch your status ... 49
7.8 When you get a committee seat ... 50
7.9 Status values ... 51
7.10 Stake by hand ... 51
7.11 Rewards and exit ... 52
7.12 What not to do ... 53

**8 Automation ... 54**
8.1 Setup without prompts ... 54
8.2 Advertised RPC URL flags ... 55
8.3 Other scripts ... 56

**9 Troubleshooting ... 58**
9.1 Sync and node mode ... 59
9.2 Setup and preflight ... 62
9.3 Scripts and updates ... 65
9.4 Public RPC and certificates ... 68
9.5 Service and keys ... 69
9.6 Staking and exit ... 72

**10 Getting help ... 73**

# 1 Introduction

Telcoin Network is an EVM-compatible layer 1 blockchain. Its validators are GSMA-member mobile network operators, and the Telcoin Association governs the network. Consensus follows the Narwhal and Bullshark protocols, and the execution layer is reth, so every node serves Ethereum-style JSON-RPC. The public testnet is Adiri, chain ID 2017. The setup script lists mainnet as coming soon. Networks and endpoints (page 10) lists the chain IDs, public RPCs and explorers.

Anyone can run a node. You need no approval to install one, follow consensus and serve JSON-RPC. Validating is a separate, on-chain step. Only GSMA-approved mobile network operators can take it, and only after the Telcoin Association approves them.

The install is the same for everyone. Every node is provisioned validator-capable, and nothing in the install decides whether it validates. What separates a node that follows consensus from one that validates is on-chain state: a stake, an activation and a seat in the committee. The protocol reads that state each epoch and derives the node's role from it.

## 1.1 What this guide covers

This guide takes one Linux server from a fresh install to a synced Telcoin Network node and keeps it running. For operators the Telcoin Association has approved, it then walks through staking and activation as a validator.

Everything runs through the scripts in the deployment repository, https://github.com/Telcoin-Association/tn-node-deployment. You run them on the node itself. The validator steps add Foundry `cast` commands, which you run from the machine that holds your wallet. The repository's README is the reference for every script option. This guide is the path through them, in the order you need them:

- Before you start (page 7): hardware, operating system, ports, networks and what to have ready.
- Install (page 12): fetch and check the scripts, run setup, see how the passphrase is stored and where files land, back up the keys, set up the firewall, check the host hardening, add the optional host baseline, and choose the optional node flags and the log target.
- Confirm the node is syncing, then synced (page 25): how to tell a healthy node from a stuck one, how long sync takes, and what the health check says about epochs and the committee.
- Day-2 operations (page 27): script and node updates, one-way updates, restarts around the epoch boundary, configuration changes, service commands, the Node Manager UI, testnet add-ons, the keys over the node's life, and removal.
- Public RPC (https and wss) (page 42): serve JSON-RPC and WebSocket on your own DNS name.
- Validators: stake and activate (page 45): checking the node with `prepare-stake.sh`, staking, activation, committee seats, rewards and exit.
- Automation (page 54): run setup and the other scripts without prompts.
- Troubleshooting (page 58): symptoms, likely causes and fixes.
- Getting help (page 73): what to send when you ask for support.

The first five chapters apply to every operator. The validator chapter applies once the Association has approved you.

## 1.2 Contact

Email devs@telcoin.org with any question about running a node. The same address handles validator onboarding, approval and hardware. If you plan to validate, write before you buy hardware.

## 1.3 Vocabulary

| Term | Meaning |
|---|---|
| Node | One `telcoin-network` install on one server, run by the `telcoin` systemd unit. |
| Epoch | The period between committee changes. Testnet epochs are six hours long. |
| Committee | The validators that vote on consensus during one epoch. |
| Validator | A node whose execution address has staked and been activated in the ConsensusRegistry contract. It votes only while it holds a committee seat. |
| Observer | Any node that is not in the current committee. |
| Execution address | The `0x` address you enter at key generation. It stakes, receives rewards, and is signed into the node's proof of possession. |
| BLS key | The node's consensus signing key, stored in `node-keys/` and encrypted with your BLS passphrase. |
| `node-info.yaml` | The node's public identity: BLS public key, proof of possession, P2P addresses and any advertised RPC URLs. |
| ConsensusNFT | A non-transferable token the Association mints to an approved execution address. Staking requires it. |

## 1.4 Roles are decided on-chain

The protocol derives each node's role every epoch from committee membership. A node that is not in the current committee is an observer, whether or not it has staked. There is no setting in the install that makes a node a validator or an observer.

Observers come in two shapes:

- A follower keeps its RPC on `127.0.0.1` and serves only the box it runs on.
- A public-RPC observer also serves `https://` and `wss://` on its own DNS name through Caddy, and advertises those URLs in `node-info.yaml` so wallets and gateways can find it (see Public RPC (https and wss) (page 42)).

Becoming a validator takes four steps, in this order:

1. Association approval, which ends with a ConsensusNFT minted to your execution address (see Prerequisites (page 45)).
2. Stake (see Send the stake (page 47)).
3. Activation (see Activate (page 47)).
4. Committee selection (see When you get a committee seat (page 50)).

The node software does not change between them.

The node reports what it is doing through the `tn_nodeMode` JSON-RPC method:

| `tn_nodeMode` | Meaning |
|---|---|
| `CvvActive` | Voting in the current committee. |
| `CvvInactive` | A committee member that is catching up before it can vote, for example after a long restart. This should be brief. |
| `Observer` | Not in the current committee. Every follower, every public-RPC observer, and every staked validator between seats reports this. |

# 2 Before you start

## 2.1 Hardware per role

These figures come from the hardware requirements page in the telcoin-network documentation. CPU counts are physical cores. Cloud vCPUs are usually hyperthreads, so an 8 vCPU instance has about 4 physical cores.

| Role | Minimum | Recommended |
|---|---|---|
| Observer, follower | 2 cores, 8 GB RAM | 4 cores, 16 GB RAM |
| Observer, public RPC | 4 cores, 16 GB RAM | 8 cores, 32 GB RAM |
| Validator | 8 cores, 32 GB ECC RAM | 16 cores, 64 GB ECC RAM |

Storage is the same for every role: at least 2 TB of TLC NVMe (10,000 write IOPS, 300 MB/s), 4 TB recommended. Avoid QLC drives; they wear out under continuous chain writes.

Validators have two network requirements on top of that:

- 200 Mbps symmetric bandwidth at minimum, 1 Gbps recommended.
- A p95 round-trip time well below 1 second to at least 7 of the 10 committee members.

> **IMPORTANT**
>
> Email devs@telcoin.org with your planned hardware before you buy anything for a validator.

The same documentation page recommends turning swap off and running time sync (chrony or systemd-timesyncd) on every node. A validator holds its vote on a header timestamped ahead of its own clock until that lead has passed, on testnet today for a lead of up to one second. It rejects a header more than five to six seconds ahead, its drift tolerance plus the five-second vote timeout; a header between the two gets no vote yet, and its proposer retries. The deployment scripts check neither, so set both up yourself.

The same page also recommends putting the data directory on ext4 or xfs and leaving the node's address-space limit unlimited, and, on bare metal, turning off CPU and NVMe power saving:

- ext4 or xfs: the node reserves disk space with `fallocate` as its storage files grow, so on those filesystems a full disk fails a write and the node stops cleanly. On a filesystem without `fallocate` (ext2, ext3, NFS, many FUSE filesystems) a full disk can crash the node, and btrfs or ZFS do not guarantee the space. The scripts do not check the filesystem.
- Address space: set no `ulimit -v` and no systemd `LimitAS=` for the node. From the next release the consensus database reserves about 62 GiB of address space, which is neither RAM nor disk, so leave the address-space limit unlimited. The unit setup writes sets no such limit.
- Power saving, on bare metal: the `performance` CPU governor and `nvme_core.default_ps_max_latency_us=0`. Cloud servers usually cannot change these, and the scripts do not check them.

`setup-node.sh` runs a hardware check during preflight and prints one line per role, for example `[OK] Full node (follower): meets minimum` or `[WARN] Validator (minimum): below minimum`. The check only warns; it never blocks setup. It counts physical cores where the server reports them, so an 8 vCPU cloud instance with 4 physical cores shows as below the validator minimum. Its `Detected:` line says what it counted: `4 physical cores`, or `8 logical CPUs` plus a note when only the logical count can be read. It does not check for ECC memory or NVMe.

## 2.2 Operating system

Use Ubuntu 24.04 LTS. A mainnet node installs only on systemd 255 or newer, which means Ubuntu 24.04. On testnet, systemd 250 or newer is enough to keep the BLS passphrase encrypted on disk (see How setup stores the passphrase (page 15)). `setup-node.sh` reads the systemd version during preflight, once you have picked the network:

| systemd | For example | What setup does |
|---|---|---|
| 255 or newer | Ubuntu 24.04 | Installs on every network. |
| 250 to 254 | Debian 12 | Installs on testnet, with the passphrase as an encrypted credential. Refused on mainnet. |
| 247 to 249 | Ubuntu 22.04 | Installs on testnet with a warning: the passphrase stays in a root-only file, `/etc/telcoin/bls-passphrase`. Refused on mainnet. |
| Older than 247 | | Refused on every network. |

A TPM 2.0, or the vTPM most cloud providers offer, lets setup bind the passphrase credential to the server. A mainnet install stops without one unless you pass `--allow-no-tpm`.

`setup-node.sh` does not support macOS.

A few parts assume a Debian-family system:

- Building from source installs its build dependencies with apt.
- A Docker install that finds no Docker installs it from Docker's apt repository, on Ubuntu or Debian only.
- `install-caddy.sh` (public RPC and the public dashboard) installs Caddy from its apt repository.
- `ui/install-ui.sh` installs Python with apt-get when Python 3 is missing.

Preflight also refuses to continue until the CVE-2026-31431 ("Copy Fail") kernel mitigation is in place. The check passes when a file under `/etc/modprobe.d/`, `/lib/modprobe.d/` or `/run/modprobe.d/` contains `install algif_aead /bin/false` and the `algif_aead` module is not loaded. Apply the mitigation from https://copy.fail before you start; you may need to unload the module or reboot. The README has the detection commands.

## 2.3 Ports

| Port | Protocol | Open to | Purpose |
|---|---|---|---|
| 49590 | UDP | Everywhere, inbound | Primary P2P (QUIC). Every node. |
| 49594 | UDP | Everywhere, inbound | Worker P2P (QUIC). Every node. |
| 443 | TCP | Everywhere, inbound | Caddy, only when it serves public RPC or the dashboard. |
| 80 | TCP | Everywhere, inbound | Caddy certificate issuance and the http-to-https redirect. Recommended with Caddy. |
| 8545 | TCP | Loopback only | HTTP JSON-RPC. Never open it. |
| 8546 | TCP | Loopback only | WebSocket JSON-RPC. Never open it. |
| 9101 | TCP | Loopback only | Prometheus metrics. |
| 43174 | TCP | 104.155.184.201/32, plus any monitoring hosts you add | Health-check endpoint, a testnet add-on. The node listens on it only when you enable the health monitor. On `v0.16.0-adiri` it listens on every interface; from the next release, given the port alone as the scripts write it, it listens on `127.0.0.1` only (see Testnet add-ons (page 38)). |
| 22 (or your SSH port) | TCP | Your admin IPs (see Firewall (page 20)) | SSH. |

Open the two UDP ports on every node, including nodes that may never validate. A node that stakes later behind a closed port cannot reach the committee and misses consensus without any obvious error. The UDP ports you open must match the ports in the multiaddrs you advertise at key generation. 49590 and 49594 are conventions, not something the node enforces: it uses the ports in its own multiaddrs. If you set the node up on other ports, open those instead; `check-node.sh` and `firewall-setup.sh` (`1) View current firewall status`) show the ports the node really uses, every worker included.

Open them in every layer that filters traffic: `ufw` on the host (see Firewall (page 20)), the cloud provider's firewall or security group, and the router's port forwards if the server is behind NAT.

## 2.4 Have these ready

- The execution address (`0x...`) the node will be registered under. For a validator this is the address that stakes and receives rewards, so it should live in a hardware wallet. It is signed into the proof of possession at key generation. Until it stakes, `prepare-stake.sh --rotate-address` can switch the node to another address (see Change the execution address before you stake (page 49)); after that it cannot change.
- A BLS passphrase, stored somewhere offline. It may not be empty, hold a line break, or start or end with whitespace, and on mainnet it needs at least 16 bytes. Setup keeps an encrypted copy on the server for the node's starts; your offline copy is what restores the keys anywhere else.
- The server's public IP, and whether it sits behind NAT. Setup detects the public IP with api.ipify.org and the internal IP from the network interface, and builds the default multiaddrs from them. Correct them at the prompts if they are wrong.

- A mounted data drive with an `/etc/fstab` entry, if the chain data will not live on the boot disk.
- A DNS A record for the node's public name, if you want public RPC (see Public RPC (https and wss) (page 42)).
- Somewhere off the server to keep a backup of the node keys (see Back up the keys now (page 19)).
- The addresses or ranges you administer the server from, to limit SSH to them (see Firewall (page 20)).
- For validators: an approval request with the Association in progress (devs@telcoin.org), and a machine with Foundry's `cast` and your wallet.

## 2.5 Networks and endpoints

| Network | Chain ID | Public RPC | Explorer |
|---|---|---|---|
| Adiri testnet | 2017 (`0x7e1`) | `https://rpc.adiri.tel` | `https://telscan.io` or `https://www.telscan.xyz` |
| Mainnet | 487 (`0x1e7`) | `https://rpc.telcoin.network` is planned and set at launch | Not launched yet |
| Devnet | 32285 (`0x7e1d`) | `https://rpc.devnet.telcoin.network` | None |

Mainnet has not launched, and two of its settings are still open in the scripts:

- the SHA-256 pins of the mainnet `genesis.yaml`, `committee.yaml` and `parameters.yaml`;
- the public RPC, which is set at launch.

Until a scripts release fills them in, a mainnet install stops: the chain-config check refuses files it has no pin for, and without the pins no mainnet release can be verified, so a Docker install is refused before anything on the server changes. The mainnet image repository is set: `ghcr.io/telcoin-association/telcoin-network`, the same as testnet's. The interactive network menu marks mainnet as coming soon.

Mainnet cannot be selected in this scripts release. One switch in the scripts' shared library, `TN_MAINNET_ENABLED` in `lib/common.sh`, stays 0 until launch, and while it is 0 every way to a mainnet install refuses before anything on the server changes:

- The setup menu's `2) Mainnet` prints `Mainnet has not launched yet.` and returns you to the menu.
- `setup-node.sh --json --network mainnet` stops with `Mainnet is not enabled in this script release (lib/common.sh TN_MAINNET_ENABLED); nothing on this host was changed.`
- The Node Manager UI keeps its Mainnet card disabled, and its helper refuses a mainnet setup.

The scripts release that launches mainnet sets the switch to 1, together with the chain-config pins and the oldest mainnet node release the scripts accept; `update-scripts.sh` brings it (see Keep the scripts current (page 27)). Setting the switch by hand gains nothing: without the pins, the chain-config check still refuses a mainnet install.

The public RPCs sit behind a load balancer and answer JSON-RPC over HTTPS only; a WebSocket upgrade gets HTTP 405. For `wss://`, use a node that serves public RPC (see Public RPC (https and wss) (page 42)).

`https://rpc.telcoin.network` is reserved for mainnet, which has not launched, and today it answers for the testnet (chain 2017). Before you treat an endpoint as a given network, check its chain ID:

```
curl -s -X POST -H 'content-type: application/json' \
  --data '{"jsonrpc":"2.0","id":1,"method":"eth_chainId","params":[]}' \
  https://rpc.telcoin.network
```

`"result":"0x7e1"` is the testnet, `0x1e7` mainnet and `0x7e1d` devnet. The scripts run this check themselves: when an RPC serves another chain than the node's, `check-node.sh` warns and skips its network comparison, and `prepare-stake.sh` stops.

Setup records the node's network in `/etc/telcoin/.node-meta` as `NETWORK=testnet`, `mainnet` or `devnet`. `check-node.sh`, `prepare-stake.sh` and the chain-config refresh in `edit-config.sh` take the network, and from it the public RPC, from that line.

# 3 Install

## 3.1 Get the scripts

Run the installer as your normal user, without `sudo`, so the scripts land in your own home directory at `~/telcoin-node-scripts`:

```
curl --proto '=https' --tlsv1.2 -fsSL https://install.telcoin.network | bash
```

The installer needs `curl` or `wget`, `tar`, `sha256sum` (or `shasum`) and `ssh-keygen` from OpenSSH 8.2 or later. If `~/telcoin-node-scripts` already exists, it asks `Re-install and overwrite? [y/N];` `| bash -s -- --yes` answers yes without asking. It moves the old tree aside first and puts it back if the new one does not install. To pick up newer scripts on an existing install, run `bash ~/telcoin-node-scripts/update-scripts.sh` instead (see Keep the scripts current (page 27)).

## 3.2 How the scripts and releases are checked

Nothing the scripts install is taken from a branch on trust.

- The scripts come from a signed release tag, `scripts-vX.Y.Z`. Each release carries a `MANIFEST` with the SHA-256 of every file and a signature, `MANIFEST.sig`. The installer holds the release signing key and the tag it installs. It checks the signature with `ssh-keygen -Y verify`, then every file against its manifest hash, and installs nothing if one check fails. `update-scripts.sh` checks each later release the same way against `lib/allowed_signers` in your install.
- Node releases are checked against the GPG keys in `lib/release-keys/*.asc`, which arrive with the signed scripts. A release publishes `SHA256SUMS` and its signature `SHA256SUMS.asc`. The scripts run `gpgv` on them and accept the release only when a key they ship signed it. A bad, expired or revoked signature refuses it; a further signature by a key that has not shipped yet only draws a warning. `SHA256SUMS` lists the binary tarball, the Docker image digest (`IMAGE_DIGEST`) and the release's three chain-config files, and the scripts check what they use against it.
- The node runs release tags only: `vX.Y.Z-adiri` on testnet, `vX.Y.Z` on mainnet. A branch, a commit id or `latest` is refused at setup and at every update. On testnet a release candidate, `vX.Y.Z-adiri-rcN`, runs only when you name its tag; no default picks one.
- Setup, `update-node.sh` and the image change in `edit-config.sh` hold every release to the oldest one the scripts support on the network, `v0.13.0-adiri` on testnet; mainnet gets its floor at launch. An older release is refused before the node changes.
- Docker installs run the image by digest (`<repository>@sha256:<digest>`), so a tag that moves later cannot change what the node runs.
- On mainnet a Docker image is named by its release tag, and the digest comes from the signed release. Testnet and mainnet images share one repository, so a digest given alone does not say which network it belongs to: setup, `update-node.sh` and `edit-config.sh` take one on mainnet only when `.node-meta` records it as the image of a mainnet release the node has installed.
- The chain configs (`genesis.yaml`, `committee.yaml` and `parameters.yaml`) are checked against SHA-256 pins that ship with the scripts before setup copies them.

## 3.3 Run setup

```
sudo bash ~/telcoin-node-scripts/setup-node.sh
```

Setup is interactive and runs in the order below. Most prompts show their default in brackets; press Enter to accept it. Yes/no prompts default to No.

1. Welcome. Answer `y` to `Ready to begin node setup?`.
2. Preflight ("Step 1 of 8"). Setup checks that it runs as root, detects the distribution, stops package updates from restarting the node or rebooting the host, and runs the CVE-2026-31431 gate. It lists your mounted filesystems and physical disks, then asks `Data directory [/var/lib/telcoin]`. Enter a path on your data drive if you have one. It then runs the hardware check, an internet check and a port check (in-use ports only warn), and installs `curl` and `git` if missing.
3. Network. Choose `1) Adiri Testnet`. `2) Mainnet` is marked coming soon and returns you to the menu. Devnet is not offered interactively. Setup then checks the systemd version against the network's floor (see Operating system (page 8)).
4. Testnet add-ons (testnet only). Each is off by default: the health-monitor endpoint on TCP 43174, log shipping, metrics shipping, a region label if you turned either shipping option on, and VPN admin SSH. Read the testnet add-ons guide before you opt in.
5. Install method (see Install methods (page 15)). A source build first asks `Install missing packages now?` if build dependencies are missing (answer y; No exits), then asks for a release tag, defaulting to the newest `-adiri` release; a branch or a commit is refused and asked again. A signed release asks `Release tag to install`. Docker asks for the image as a release tag or a digest, defaulting to the newest release resolved to its digest. "I already have it" first offers a binary it finds (`Use this binary?`) and asks `Full path to telcoin binary:` only if it finds none or you decline. The build, download or pull happens here, so a source build holds this step for 20 to 40 minutes; stay attached to the session.
6. Passphrase storage. Setup does not ask: it prints how it will keep the BLS passphrase, for example `BLS passphrase: encrypted systemd credential (host+tpm2)` (see How setup stores the passphrase (page 15)). On mainnet, a server with no usable TPM stops here unless you passed `--allow-no-tpm`.
7. Node configuration ("Step 3 of 8"). First, RPC access: `1) Private` keeps RPC on `127.0.0.1`; `2) Public` asks for the node's DNS name, then always asks for an optional inbound public IP (Enter auto-detects; set it behind NAT; see Public RPC (https and wss) (page 42)). Then the ports: P2P primary `49590`, P2P worker `49594`, RPC `8545`, metrics `9101`. Then `Use these default paths?` for the config, log and install directories, and an optional advertised node name.
8. System infrastructure ("Step 4 of 8"). Service user and group, both `telcoin` by default. The service user gets no login shell and may not belong to an admin group, and setup adds a `telcoin-log` group for people who read the log files. Source and signed-release installs also get a `tn-build` account, which builds or checks the binary so that root never runs it first. Docker and existing installs may be asked `Clone the repository now to get the chain-config files?`. Answer y, because No means copying the chain configs by hand before step 10. If you passed node flags (see Optional node flags (page 23)), setup asks the installed release here whether it has them, and stops before any key is made if it does not.
9. Keys ("Step 5 of 8"). If `node-keys/` already exists, setup asks `Overwrite existing keys?`. On a re-run, answer No (see What not to do (page 53)). Otherwise it asks for the execution address and prints the detected public IP (it asks for one only if detection fails). It then asks for the external primary and worker

addresses and, after printing the detected internal IP, the listener primary and worker addresses. Each shows a default built from the detected IP, such as `/ip4/<public-ip>/udp/49590/quic-v1`; check the IP in them. Then it asks for the BLS passphrase twice. The keytool writes the keys and `node-info.yaml`, setup stores the passphrase, prints `node-info.yaml` and the staking steps (with the live stake amount and a pointer to `prepare-stake.sh`), and waits at `Press Enter to confirm you have backed up your keys`. Do the backup in Back up the keys now (page 19) before you press Enter.

10. Configuration ("Step 6 of 8"). Checks `genesis.yaml`, `committee.yaml` and `parameters.yaml` against the pinned hashes and copies them into the data directory. If it cannot find them, it prints where to get them and waits for you to copy them in.
11. Service ("Step 7 of 8"). Writes `/etc/telcoin/node.env`, the start script, the systemd unit and `/etc/telcoin/.node-meta`, then asks `Start the node now?`. Only if you answer yes and the service comes up does it ask `Enable auto-start on server reboot?`. Answer yes to both.
12. Public RPC, only if you gave a domain and the node started. Setup checks DNS and enables the endpoint. If DNS is not ready yet, setup still finishes and prints the exact command to run later.
13. The testnet add-ons you opted into are applied. Log or metrics shipping asks for your ingest token (hidden; leave it blank to skip). With `--host-baseline`, the host baseline comes next (see Host baseline (optional) (page 22)).
14. A summary with the paths, how the passphrase is kept, the P2P ports `node-info.yaml` advertises (every worker), a firewall reminder for those ports, the next steps and a link to the operator runbook.

`sudo bash ~/telcoin-node-scripts/setup-node.sh --help` lists the public RPC flags, the install and release flags, the node flags, and the passphrase and log options. An argument setup does not recognise is reported on stderr as `Unknown argument: <arg>`, and setup carries on without it, so check the spelling of any flag you pass. Public RPC (https and wss) (page 42) covers the public RPC flags, Optional node flags (page 23) the node flags, Logs (page 24) the log options, and Automation (page 54) the flags for unattended runs.

## 3.4 Install methods

| Menu choice | `--install-method` | What it does | How it updates |
|---|---|---|---|
| `1) Build from source` | `source` | Installs the build dependencies with apt, clones telcoin-network to `/opt/telcoin-source`, checks the release tag's signature, and builds it as the `tn-build` account with a pinned Rust toolchain (20 to 40 minutes). Every `tn-build` process is stopped after each build command, before root copies the output. Root only installs the result, as `/opt/telcoin/telcoin-network`. | `update-node.sh` |
| `2) Signed release` | `release` | Downloads the release tarball with `SHA256SUMS` and `SHA256SUMS.asc`, checks the signature and the hashes, runs the binary's `--version` as `tn-build`, and installs `/opt/telcoin/telcoin-network`. | `update-node.sh` |
| `3) Docker` | `docker` | Installs Docker if missing, from Docker's apt repository after checking its signing key, and turns on `live-restore`. Pulls the image by digest from `ghcr.io/telcoin-association/telcoin-network` (on testnet also the older `us-docker.pkg.dev/telcoin-network/tn-public/adiri`) and checks the pulled digest. The default is the newest release published on GitHub, by the digest its signed release names. Until testnet has one (signed releases are planned to start with `v0.17.0-adiri`), the default is the newest image of the older repository, or `v0.16.0-adiri` if that cannot be read, with a warning. | `update-node.sh` |
| `4) I already have it` | `existing` | Uses a `telcoin-network` binary you supply. | By hand (see Update the node (page 28)). |

`--release-tag <tag>` picks the release for the first three; `--build-ref` is its older name. On testnet a source build of a tag whose signature cannot be checked goes ahead with a warning; on mainnet it is refused. Release tarballs are published for x86_64 Linux; on another architecture, build from source or use Docker. A failed source build keeps its log in `/var/lib/telcoin-staging/`. The README has more on install options and network binding.

## 3.5 How setup stores the passphrase

The node needs the BLS passphrase each time it starts, to unlock its key. Setup keeps it on the server as an encrypted systemd credential, `/etc/telcoin/bls-passphrase.cred` (root, mode 600), made with `systemd-creds encrypt` and decrypted once as a check before it is put in place. The plaintext is not written to disk. The credential is encrypted with the host's key and, when the server has a usable TPM 2.0, with the TPM as well; `.node-meta` records which as `CREDENTIAL_KEY=host+tpm2` or `CREDENTIAL_KEY=host`. A copy of the file cannot be decrypted on another machine.

At each start, systemd decrypts the credential for the service, and the start script hands it to the node on standard input (`--bls-passphrase-source stdin`). The passphrase is never on a command line, in the unit file or in the service's environment. Root on the server can still decrypt the credential, so it protects the passphrase at rest and away from the server, not from root on the server.

| Option | Effect |
|---|---|
| none | `host+tpm2` when the server has a usable TPM, else `host`. On testnet `host` only warns. |
| `--allow-no-tpm` | Mainnet: accept `host` on a server with no usable TPM. Without it, a mainnet install stops. |
| `--bind-pcrs <list>` | Also bind the credential to these TPM PCRs, joined by `+` (for example `7`). Needs a TPM. A firmware or Secure Boot change to those PCRs stops the node from starting until you run `edit-config.sh --reseal`. |
| `--passphrase-method stdin` | Testnet and devnet; refused on mainnet. Keep the passphrase in the root-only file `/etc/telcoin/bls-passphrase` (mode 600) instead. Setup does this by itself below systemd 250. |
| `--passphrase-stdin` | Read the passphrase from standard input (one line, no echo) instead of prompting for it. |

To change the stored passphrase, or to seal the credential again after a TPM or firmware change, use `edit-config.sh` (see Change the configuration (page 34)). Neither re-encrypts the keys, so keep the passphrase the keys were generated with. On a node release whose keytool has a `verify` command, `edit-config.sh` checks that the passphrase opens the BLS key before it stores anything; on a release without it, nothing checks this.

## 3.6 Where things land

| Path | Owner, mode | What it is |
|---|---|---|
| `/opt/telcoin/telcoin-network` | root, 0755 | The node binary for source and signed-release installs. An `existing` install runs the binary from the path you gave setup (`Binary=` in its summary). |
| `/usr/local/libexec/telcoin/tn-start` | root, 0755 | Start script for source, release and existing installs. It hands the passphrase to the node on standard input and holds the node's launch command and flags. |
| `/usr/local/libexec/telcoin/tn-start-docker` | root, 0755 | Start script for Docker installs. The container runs as the service account, read-only, with every capability dropped. |
| `/etc/systemd/system/telcoin.service` | root, 0644 | The systemd unit, `telcoin`. It loads the passphrase credential and sandboxes the node. Docker installs also name the container `telcoin`. |
| `/etc/telcoin/` | root, 0755 | Configuration directory. |
| `/etc/telcoin/bls-passphrase.cred` | root, 0600 | The encrypted passphrase (see How setup stores the passphrase (page 15)). |
| `/etc/telcoin/node.env` | root, 0644 | The listener multiaddrs (`PRIMARY_LISTENER_MULTIADDR`, `WORKER_LISTENER_MULTIADDR`) and `TN_RPC_ADDR`. No secrets. |
| `/etc/telcoin/.node-meta` | root, 0600 | Install settings the other scripts read, one `KEY=value` per line, among them `NETWORK`, `INSTALL_METHOD`, `RELEASE_TAG`, `DOCKER_IMAGE` or `BINARY_PATH`, `DATA_DIR`, `RPC_PORT`, `VALIDATOR_ADDRESS`, `PASSPHRASE_METHOD`, `CREDENTIAL_KEY`, `LOG_TARGET`, `BOOTSTRAP_PEERS_FILE`, `STATE_EXPORT`, and `PUBLIC_RPC_DOMAIN`, `PUBLIC_RPC_URL` and `PUBLIC_WS_URL` while public RPC is on. Root-only, so run the scripts with `sudo`. |
| `/etc/telcoin/bootstrap-peers.yaml` | world-readable | The peers map, when the node runs with `--bootstrap-peers` (see Optional node flags (page 23)). |
| `/var/lib/telcoin/` | service user, 0700 | The data directory. |
| `/var/lib/telcoin/node-keys/` | service user | BLS and P2P keys. Back these up. |
| `/var/lib/telcoin/node-info.yaml` | service user | Public identity and advertised URLs. |
| `/var/lib/telcoin/genesis/`, `/var/lib/telcoin/parameters.yaml` | service user | Chain configuration. |
| `/var/lib/telcoin/db/` | service user | Chain database. This is what grows. |
| `/var/lib/telcoin/consensus-db/epochs/` | service user | The consensus packs of finished epochs and the pack of the epoch in progress, open for append, in `epoch-N` directories. |
| `/var/lib/telcoin/consensus-db/epoch/`, `kad/`, `cache/` | service user | On `v0.16.0-adiri`: the epoch database (only the epoch in progress, the node's own votes and proposals included), the peer records and the batch caches. |

| Path | Owner, mode | What it is |
|---|---|---|
| `/var/lib/telcoin/consensus-db/tndb/` | service user | From the next release: the same three databases, as `tndb/epoch`, `tndb/kad` and `tndb/cache`. The first start copies them over from the directories above, deletes the old files and writes the marker `tndb/mdbx-migrated` (see One-way updates (page 30)). |
| `/var/lib/telcoin/consensus-db/state_exports/` | service user | One directory per epoch when state export is on (see Optional node flags (page 23)). |
| `/var/log/telcoin/` | root:telcoin-log, 0750 | `telcoin.log` and `telcoin-error.log` (0640) when the node logs to files (see Logs (page 24)). |
| `/var/log/telcoin/node/` | service user:telcoin-log, 0750 | Logs the node writes itself when testnet log shipping is on. |
| `/var/backups/telcoin/` | root, 0700 | The copies the scripts take before they change a file. |
| `/run/telcoin/update.lock` | root, 0600 | The update lock: one script at a time changes the node. |
| `/var/lib/telcoin-staging/` | root, 0700 | Downloads and build logs while they are checked. |
| `/opt/telcoin-source/`, `/var/lib/tn-build/` | root; `tn-build` | The telcoin-network source tree and the build account's home, where it builds (source builds). |

If you chose a different data directory, read it in place of `/var/lib/telcoin` everywhere in this guide. Everything outside the data directory belongs to root. The service account writes only to the data directory, and to `/var/log/telcoin/node/` when log shipping is on. `sudo bash ~/telcoin-node-scripts/check-node.sh` compares the server with this layout and reports any path that differs. The README's system layout shows the full tree.

## 3.7 Back up the keys now

> **WARNING**
>
> Losing `node-keys/` means losing the node's identity. If you have not staked yet, you can regenerate keys and carry on. Once you have staked, you have to go back to the Association.

Back up `node-keys/` and `node-info.yaml` together. Keep the BLS passphrase somewhere else (a password manager, or paper in a safe), never next to the archive. In a second terminal on the server:

```
(umask 077; sudo tar -C /var/lib/telcoin -czf - node-keys node-info.yaml \
  > ~/telcoin-keys.tgz)
```

Copy the archive off the server from your own machine, then delete it from the server:

```
scp user@SERVER_IP:telcoin-keys.tgz .
ssh user@SERVER_IP 'shred -u ~/telcoin-keys.tgz'
```

Treat the archive as a secret. Then go back to setup and press Enter.

## 3.8 Firewall

```
sudo bash ~/telcoin-node-scripts/firewall-setup.sh
```

Choose `2) Enable firewall with recommended defaults`. It shows where SSH is allowed from now and asks `Restrict SSH to these sources (comma-separated IPs/CIDRs; Enter keeps the current setting)`. Give the addresses or ranges you administer the server from: the plan then allows SSH from those sources only and removes the open SSH rule. On mainnet, Enter with no sources asks `Keep SSH open to every address anyway?`, and No cancels the enable.

The plan sets inbound to deny and outbound to allow, allows established connections and SSH, allows the node's P2P ports when a node is installed, and on testnet allows TCP 43174 from the Association monitor only. It reads the P2P ports from the node's launch command, `/etc/telcoin/node.env` and `node-info.yaml`, every worker included, so a node set up on other ports than 49590 and 49594 gets the right rules. When Caddy serves the public RPC or the dashboard on this server, it also allows TCP 80 and 443 before it turns `ufw` on, so the site stays up. The plan lists every rule before it asks `Apply recommended firewall defaults?`. It then asks whether to reset to a clean slate first; No keeps your existing rules. If a step fails, the script puts the previous rules back. Test SSH from a new terminal before you close the current one.

Without the menu, pass the sources with `--enable` (or `--reset`, which clears every rule first):

```
sudo bash ~/telcoin-node-scripts/firewall-setup.sh --enable \
  --ssh-from 198.51.100.0/24,203.0.113.7
```

- `--ssh-from` and `--p2p-from` take a comma-separated list of addresses or CIDRs, or a file with one entry per line (`#` starts a comment). A range wider than /8, `0.0.0.0`, `::` and IPv4 octets with leading zeros are refused.
- `--p2p-from` limits the P2P ports to the sources you list. Every peer and committee member that dials the node must then be on it, so most nodes leave it out.
- `--ssh-anywhere` and `--p2p-anywhere` drop a stored list and open the port to every address again.
- The lists are stored in `.node-meta` as `SSH_ALLOW_FROM` and `P2P_ALLOW_FROM` and reused on later runs. A new list replaces the old one, and the rules for sources it drops are deleted.
- On mainnet, an enable with no SSH list stored or given stops with `Mainnet: give the SSH sources with --ssh-from CIDR[,CIDR] (or --ssh-anywhere to keep SSH open to every address); nothing was changed.`
- On Google Cloud, the IAP range `35.235.240.0/20` joins the SSH sources, so the console's SSH button keeps working.

When the P2P ports stay open to every address (no `--p2p-from` list), every enable, the menu's included, also limits new P2P flows per source. One IPv4 address, or one IPv6 /64, may send at most 20 packets of new, not yet answered UDP flows to the P2P ports per minute, after a burst of 40; packets above that are dropped. A flow the node has answered is never limited, so peers the node already talks to are not slowed. The limit is an nftables table, `inet tn_p2p`, that runs before `ufw`, only drops, and leaves every other decision to `ufw`. The enable prints it, for example `P2P new-flow limit: UDP 49590 and 49594 accept at most 20 packets of new, not yet answered flows per minute from one IPv4 address or IPv6 /64 after a burst of 40 (nftables table inet tn_p2p).` It also writes `/etc/telcoin/p2p-limit.nft` and enables `telcoin-p2p-limit.service`, which loads the table again at every boot, after `ufw`.

- With a `--p2p-from` list the limit is not used, and the enable removes it: the list is the stricter rule.
- Without nftables (`nft`), the enable warns that the limit is not applied and sets up the `ufw` rules alone.
- `TN_P2P_NEW_FLOW_RATE` and `TN_P2P_NEW_FLOW_BURST` (whole numbers from 1 to 100000) change the 20 and 40 for one run. Pass them through `sudo`, for example `sudo TN_P2P_NEW_FLOW_RATE=30 TN_P2P_NEW_FLOW_BURST=60 bash ~/telcoin-node-scripts/firewall-setup.sh --enable`. A value outside that range stops the run before anything changes.
- The two values are not stored, so a later enable without them, the Node Manager UI's included, goes back to 20 and 40.
- `1) View current firewall status` shows a `New-flow limit:` line under the P2P ports, and whether the limit comes back at boot.

`3) Manage SSH access` hardens sshd through a drop-in, `/etc/ssh/sshd_config.d/00-telcoin-hardening.conf`. `1) Disable password authentication (keys only)` writes `PasswordAuthentication no` and `KbdInteractiveAuthentication no`, and `2) Disable root login` writes `PermitRootLogin no`. The script checks the result with `sshd -t` and puts the previous configuration back if sshd rejects it. It warns when `/etc/ssh/sshd_config` has no `Include /etc/ssh/sshd_config.d/*.conf` line, because sshd then ignores the drop-in. Confirm that your key login works before you turn passwords off.

`ufw` is only the host layer. Open the same UDP ports in the cloud firewall or security group as well, and forward them on the router if the server sits behind NAT. `1) View current firewall status` lists each P2P port with its label, for example `UDP 49594 is open (worker)`. The README covers the rest of the firewall script.

## 3.9 Check the host hardening

```
sudo bash ~/telcoin-node-scripts/check-node.sh --audit
```

`--audit` changes nothing and prints one `ok`, `warn` or `info` line per check:

- the systemd version, against the version encrypted credentials need;
- how the passphrase is stored, the credential's owner and mode, and any plaintext passphrase file left beside it;
- a passphrase held in the unit;
- whether systemd can use a TPM;
- disk encryption under the data and config directories;

- core dumps (`LimitCORE=0` on the unit, `fs.suid_dumpable`, `kernel.core_pattern`) and `kernel.yama.ptrace_scope`;
- the host baseline (see Host baseline (optional) (page 22)).

It exits 0 when nothing warned and 1 otherwise. `setup-node.sh --audit` prints the same summary and exits, so you can run it before an install.

## 3.10 Host baseline (optional)

The host baseline records who touches the node's keys and which of its files change. It is off by default, and setup never asks for it:

```
sudo bash ~/telcoin-node-scripts/setup-node.sh --host-baseline
```

On a server whose node is already installed, that command makes the baseline, or makes it again, and changes nothing else. Added to a new install, it runs after the node, public RPC and the testnet add-ons are in place; in a `--json` run it takes effect at finalize, and the `done` event carries `"host_baseline":true` or `false`. It needs apt, so it works on Ubuntu and Debian only.

It does four things:

1. Installs `auditd` and `aide` with apt, without their recommended packages, so neither a daily whole-system AIDE check nor a mail server comes with them.
2. Writes the audit rules `/etc/audit/rules.d/telcoin-node.rules` and loads them. The kernel's audit log then records every access to `<data dir>/node-keys` (key `telcoin-keys`) and every write to `/etc/telcoin` (`telcoin-config`) and to the node's unit file (`telcoin-unit`).
3. Writes its own AIDE configuration, `/etc/aide/telcoin-node.conf`, for `/opt/telcoin`, the start scripts in `/usr/local/libexec/telcoin`, `/etc/telcoin` and the unit, and makes the baseline database `/var/lib/aide/telcoin-node.db`. The passphrase files and the ingest token are left out, because they change when you rotate them.
4. Records `HOST_BASELINE=true` in `.node-meta`.

A step that fails only warns, `The host baseline was not made (see above); the node runs without it. Try again with: sudo ./setup-node.sh --host-baseline`, leaves nothing of the baseline behind (no rules, no AIDE configuration, no marker), and setup goes on.

The scripts make the AIDE baseline again after each change they apply, so their own writes do not show up as differences:

- `update-node.sh` after an update that passed its checks;
- `edit-config.sh` after an edit whose restart came back healthy, after `12) Restart node`, and after a passphrase change, `--reseal` or `--migrate-credential`;
- `install-caddy.sh`, `firewall-setup.sh`, `setup-vpn.sh` and `setup-observability.sh` after a change that went through.

A refused or failed change, or a rollback, leaves the baseline as it was. Nothing runs the check on a schedule, so run it yourself. A refresh takes in every file as it is at that moment, so to see a change made since the last refresh, check before you update or edit:

```
sudo aide --config=/etc/aide/telcoin-node.conf --check
sudo ausearch -k telcoin-keys
```

`aide --check` exits 0 when nothing changed and otherwise lists each file that did. `ausearch -k telcoin-keys` lists the recorded accesses to the keys; `-k telcoin-config` and `-k telcoin-unit` do the same for the configuration and the unit.

`check-node.sh --audit` shows the baseline's state. Without it the line reads `host baseline: off (sudo ./setup-node.sh --host-baseline adds auditd watches and an AIDE baseline)`. With it, the audit shows `HOST_BASELINE=true`, the rules file, whether the kernel has the rules loaded (only as root) and how many days old the AIDE database is; a missing piece is a `warn` line that names the command to make it again. `remove-node.sh` removes the rules, the AIDE configuration and database and the marker together with the last node on the server; the `auditd` and `aide` packages stay, and `sudo apt-get purge auditd aide` removes them.

## 3.11 Optional node flags

Setup can pass two extra features to the node. Both are optional, and `edit-config.sh` can add, change or remove them after the install (see Change the configuration (page 34)).

`--bootstrap-peers <file>` replaces the bootstrap servers in the genesis with your own list of peers to dial at start. It needs `v0.15.0-adiri` or later. The node treats these peers as discovery hints only: it does not keep them connected ahead of other peers or exempt them from load penalties.

- The file is a YAML or JSON map keyed by BLS public key. Each value is that node's `primary` and `workers` entries, copied from under `p2p_info` in its `node-info.yaml`. The README's bootstrap peers section shows the shape.
- It must be a regular file of at most 64 KiB that other users can read (`chmod 644`). The map does not stay private: setup installs it world-readable as `/etc/telcoin/bootstrap-peers.yaml`, and its check passes the map on the node binary's command line. A file its group or other users cannot read is refused, with the `chmod` to run.
- Setup checks the map with the release's own parser and stops with the parser's reason if a key or value is wrong.
- The start script passes `--bootstrap-peers "$(cat /etc/telcoin/bootstrap-peers.yaml)"`, so the node reads the file again at every start. An edit to that file takes effect at the next restart without any check, and if the file goes missing or empty the node does not start. `edit-config.sh` checks a new map before it installs it.

`--enable-state-export` (`v0.13.0-adiri` or later) writes each epoch's final execution state under `<data dir>/consensus-db/state_exports/epoch-N/`. Every export is a full copy of the state and none is deleted, so the disk fills over time. `--state-export-keep <n>` keeps only the newest n exports (1 to 999999) and turns export on by itself; it needs `v0.15.0-adiri` or later. A value of 2 or more keeps the previous export while the next one is written.

```
sudo bash ~/telcoin-node-scripts/setup-node.sh \
  --bootstrap-peers ./peers.yaml --state-export-keep 3
```

Setup asks the release it installs whether it has each flag you give, during System infrastructure ("Step 4 of 8"), and stops before any key is made when it does not. It records the choices in `.node-meta` as `BOOTSTRAP_PEERS_FILE` (the installed path, empty when none) and `STATE_EXPORT` (`off`, `unlimited` or a number).

The node also reads `network-config`, a YAML file in the data directory; the scripts and the Node Manager UI write only its `hostname` line, the node name it advertises. From the next release the node accepts six opt-in keys there: `process_budget`, `source_admission`, `trusted_nodes`, `committee_peers`, `admission` and `serve_budget`; `v0.16.0-adiri` ignores them. They are unset by default, which means absent from the file, and no script sets them. Leave them unset unless devs@telcoin.org asks you to set one. The `source_admission` key can refuse committee peers once its limits fill, and a node that has it set logs a warning at start that begins `source admission is enabled`.

## 3.12 Logs

`--log-target` tells setup where the node's output goes:

- `file`, the default on testnet: `/var/log/telcoin/telcoin.log` and `telcoin-error.log`, rotated by `/etc/logrotate.d/telcoin` once a file passes 100 MB, with five old files kept. Members of the `telcoin-log` group can read them without `sudo`.
- `journal`, the default on mainnet: the systemd journal only. Setup caps the journal with `/etc/systemd/journald.conf.d/50-telcoin.conf` (`SystemMaxUse=4G`, `SystemKeepFree=2G`) and keeps an existing file of that name.

`journalctl -u telcoin` shows the service's own messages either way. Setup records the choice as `LOG_TARGET` in `.node-meta`.

# 4 Confirm the node is syncing, then synced

Check that the service is up and watch its log for a minute:

```
systemctl status telcoin
journalctl -u telcoin -f
```

Then run the health check:

```
sudo bash ~/telcoin-node-scripts/check-node.sh
```

It compares your node with the public RPC of the network in `.node-meta` (`https://rpc.adiri.tel` on testnet; see Networks and endpoints (page 10)), or with the RPC you pass as `--network-rpc <URL>`, and ends with one verdict:

- `CATCHING UP: <N> execution blocks behind` while the node is more than 50 blocks behind the network.
- `All checks passed -- node is healthy and caught up (EVM lag <N>)` once it has caught up.
- `<N> issue(s) found -- review warnings/errors above` when something else is wrong.

Run it twice a few minutes apart. The second run also reports how far the local block advanced since the first, which tells you whether the gap is closing. The report header has a `Network:` line with the network, its chain ID and where that came from. The first section lists what the node runs: the Docker image or binary, the bootstrap peers and state export when set, the P2P ports and the RPC URLs it advertises. When the node binary supports `tn_nodeMode`, the report also prints a `Consensus role:` line. The script always exits 0, so anything that consumes it should read the verdict line, not the exit code.

If the comparison RPC serves another chain than the node's, the report says so and skips the comparison instead of blaming the node, for example `https://rpc.telcoin.network serves chain 2017 (testnet), not chain 487 (mainnet) -- skipping the network comparison`. It does not count as an issue; `--network-rpc <URL>` with an RPC for the node's chain brings the comparison back.

## 4.1 The epoch section

The `Checking epoch and committee...` section shows the current epoch, the time to the next boundary and its UTC time, and committee membership for this epoch and the next two, for example `epoch 581: not a member · 582: not a member · 583: member`. "Not a member" is normal for a node that is not a validator, for a validator before its first seat, and for an Active validator between seats. Such a node also gets the info line `Not in recent headers -- expected: this node is not in the committee for epoch 581`. Only a committee member missing from the latest headers is an error.

For a staked validator the section adds the activation epoch and the earliest committee seat (see When you get a committee seat (page 50)). It also compares the node's workers with the number the network requires (`WorkerConfigs.numWorkers()`, 1 on testnet today); fewer is an error.

## 4.2 Checking by hand

Do not use `eth_syncing`. On Telcoin Network it returns `false` even while the node is catching up. Compare `eth_blockNumber` on your node with the public RPC instead:

```
bn() { curl -s -X POST -H 'content-type: application/json' \
  --data '{"jsonrpc":"2.0","id":1,"method":"eth_blockNumber","params":[]}' "$1" \
  | sed -E 's/.*"result":"(0x[0-9a-fA-F]+)".*/\1/'; }
echo "local:   $(( $(bn http://127.0.0.1:8545) ))"
echo "network: $(( $(bn https://rpc.adiri.tel) ))"
```

The node is synced when the two numbers stay within a few blocks of each other.

To see the node's consensus role:

```
curl -s -X POST -H 'content-type: application/json' \
  --data '{"jsonrpc":"2.0","id":1,"method":"tn_nodeMode","params":[]}' \
  http://127.0.0.1:8545
```

Any node outside the committee returns `"Observer"`, including a staked validator that has not been seated yet.

## 4.3 How long it takes

There is no published figure for a first sync; it depends on chain height, disk and bandwidth. Watch the lag shrink rather than waiting for a fixed time. Each restart replays recent consensus output before the node catches
up again, so a node that is restarted often stays behind. If the lag does not shrink between runs, see Troubleshooting (page 58).

# 5 Day-2 operations

## 5.1 Keep the scripts current

```
bash ~/telcoin-node-scripts/update-scripts.sh
```

Run it as your normal user, without `sudo`. It looks up the newest scripts release, downloads its `MANIFEST` and `MANIFEST.sig`, and checks the signature against `lib/allowed_signers` in your install. It refuses a release older than the one you have (each manifest carries a serial number), and one that reuses your serial with different content. It lists what changes and asks `Install these changes? [Y/n]`, then downloads every changed file and checks each against its manifest hash before it installs any of them. The files install together or not at all: on any refusal or failed check the tree stays as it was, and the run exits 1. New scripts arrive the same way, and files a release no longer ships are removed. `~/telcoin-node-scripts/MANIFEST` records the release you have.

When a release needs a newer updater than yours, the run stops and prints the installer command. Run it, then run the updater again:

```
curl --proto '=https' --tlsv1.2 -fsSL https://install.telcoin.network | bash
```

The updater never runs `sudo` and does not touch the node or the Node Manager UI. When UI files changed, it prints the command that loads them:

```
sudo bash ~/telcoin-node-scripts/ui/install-ui.sh --update
```

Run the updater before every node update. `update-node.sh` learns how to handle new releases through these updates.

From `scripts-v2.1.0` on, each scripts release records its release date and an expiry date in its `MANIFEST`, 90 days later unless the signer chose otherwise. An expiry date is a reminder that a newer signed release is due, not a refusal. From that day on, `update-scripts.sh` and the installer warn `Script release <tag> expired on <date>; a newer signed release is overdue. Installing it anyway.` and still install the release, which verifies as before. If no newer release appears soon after, email devs@telcoin.org.

To see where you stand without changing anything:

```
bash ~/telcoin-node-scripts/update-scripts.sh --check
```

`--check` installs nothing, needs no root and asks nothing. It prints the installed release and the newest signed one, each with its serial, release date and expiry date, says whether an update is pending, and warns when either release has expired. It exits 0 when the installed release is the newest and neither has expired, 1 when an update is pending or a release has expired, and 2 when no signed release is installed, a release cannot be fetched or does not verify, or the updater would refuse the newest one. `check-node.sh` reports the installed release too, in its `Checking the script release...` block, from the installed record alone; an expired release there is a warning, not a health issue.

## 5.2 Update the node

```
sudo bash ~/telcoin-node-scripts/update-node.sh
```

Before you apply a release, read its notes: a signed release's page on GitHub lists its files, the commands to verify them, and operational notes such as a required resync or a configuration change. The script works out how the node was installed and offers the matching versions. Signed-release installs ask `Release tag to install`, defaulting to the newest release of the network. Source installs get the same version picker as setup, defaulting to the newest release tag. Docker installs list the network's recent release tags, newest first, with the current and latest ones marked; the tag you pick is resolved to its digest and pulled by digest. Updates take release tags only: a branch, a commit or `latest` is refused. Then it asks how far to go:

- `1) Prepare only` builds, downloads or pulls the new release while the node keeps running. Apply it later in a quiet window.
- `2) Prepare AND apply` also stops the node, swaps in the new version, restarts it and checks it.
- `3) Cancel` cancels.

A prepared update survives between runs. The next run offers `1) Apply now`, `2) Discard and prepare a different update` or `3) Exit`. `sudo bash ~/telcoin-node-scripts/update-node.sh --discard` drops a prepared update after asking `Discard this pending update?`. After an apply that was cut short, a discard puts back the unit, start script and `node.env` that apply had rewritten, unless it was a one-way update that had already started the new release: then the discard keeps them as the apply wrote them and prints how to go back.

What else to expect:

- On a node registered on-chain as a validator (status 1 to 4), or one whose status cannot be read, the script shows a downtime warning and asks you to type `CONFIRM` before it stops the node.
- Before it stops the node, it runs the committee quorum check and then the epoch wait (see Restarts and the epoch boundary (page 31)). `--no-epoch-wait` skips the wait, never the quorum check.
- The apply writes the start script and the unit again from `.node-meta` while the node is stopped, so hand edits outside the node's launch line do not survive it.
- After the restart it waits up to 45 seconds (600 on a one-way update) for the service to be active and for the node to answer `tn_latestConsensusHeader`. When many nodes restart at once, quorum takes longer to form. Raise the window with `TN_UPDATE_VERIFY_TIMEOUT`, in seconds, for example `sudo TN_UPDATE_VERIFY_TIMEOUT=120 bash ~/telcoin-node-scripts/update-node.sh`. A number you set always wins, on a one-way update too, so leave it unset there or give it 600 or more.
- It then checks that the node follows consensus: the consensus header has to advance, and a node that was voting (`CvvActive`) before the update has to vote again, within `TN_UPDATE_VOTE_TIMEOUT` seconds (600 by default).
- When that window runs out while the unit is active and the node still answers `eth_chainId`, the update does not fail: catching up after the restart can take longer than the window. The new release keeps running, nothing is rolled back, and the script warns `... Not rolled back: the unit is active and its RPC answers, so the new release keeps running while it catches up. ...`. An interactive run ends with `telcoin runs the new release but is not following consensus or voting yet; it was not rolled back.`, and a `--json` apply ends `"ok":true` with `"voting":false`. Run `check-node.sh` again a few minutes later. To count this case as a failed check, which then offers the rollback (a

`--json` apply rolls back on its own), set `TN_UPDATE_VOTE_ROLLBACK=1` through `sudo`: `sudo TN_UPDATE_VOTE_ROLLBACK=1 bash ~/telcoin-node-scripts/update-node.sh`. Only the value `1` turns it on. Leave it unset for the one-way update to the next release, whose rollback you decline (see below). A wrong chain ID, a unit that is no longer active or an RPC that no longer answers fails the check whatever the setting.

- If a check fails, it offers to roll back to the previous binary or image, except after the one-way update to `v0.16.0-adiri` (next item). After the one-way update to the next release it still offers the rollback, which you decline (the item after).
- When the node runs a release older than `v0.16.0-adiri` and the target is `v0.16.0-adiri` or newer, the update is one-way. The new release migrates the consensus store on its first start, and older releases cannot open the data directory afterwards. Snapshot the data directory before you apply (see One-way updates (page 30)). The script says the update is one-way, warns when the free space under `consensus-db/epochs` is less than twice its largest `epoch-N` directory, asks whether you have a snapshot, and waits up to 600 seconds. If the check fails it does not roll back and does not stop the node, which may still be migrating, and it leaves the unit, start script and `node.env` on the new release. When the script cannot read which release the node runs, it treats the update as one-way.
- An update from `v0.16.0-adiri` to the next release or a later one is one-way too: from the next release the node keeps its consensus database in a new format, and its first start moves the database over (see One-way updates (page 30)). Snapshot the data directory before you apply, whether or not the script asks about a snapshot. On a committee node, apply this update from a terminal, without `TN_ASSUME_YES`, not from the Node Manager UI: the UI's apply rolls back on its own after a failed check, which would start `v0.16.0-adiri` with empty epoch tables. If a check fails after this update and the script asks `Roll back to previous binary?` or `Roll back to previous image (<image>)?`, answer no, and email devs@telcoin.org before you go back.
- Downgrades are refused: `<new> is older than the running <current>: downgrades are refused.` On testnet, an interactive run with `--allow-downgrade` goes ahead anyway.
- A target older than the oldest release the scripts support on the network (`v0.13.0-adiri` on testnet) is refused on every install method, before anything is fetched: `<ref> is older than v0.13.0, the oldest testnet release these scripts support (...). Pick v0.13.0 or a newer release. Nothing was changed.`
- On mainnet, a Docker update takes the release tag; a digest alone is taken only when `.node-meta` records it as the image of a mainnet release the node has installed (see How the scripts and releases are checked (page 12)).
- Only one update runs at a time. A second run stops with `Another update is already running (PID N).`
- It never touches the keys, `node-info.yaml`, the BLS passphrase, the chain configuration, or the listener addresses.

The script does not update `existing` installs. For one, it stops, names the binary the node runs (recorded as `BINARY_PATH` in `.node-meta`), and prints how to check a release by hand. On the server, as your normal user (not root), in an empty directory, with the release you want as `TAG`. Signed releases are planned to start with `v0.17.0-adiri`; `v0.16.0-adiri` and older have no release page.

```
TAG=vX.Y.Z-adiri   # a published release
base=https://github.com/Telcoin-Association/telcoin-network/releases/download/$TAG
tgz=telcoin-network-$TAG-x86_64-unknown-linux-gnu.tar.gz
curl -fsSL -O "$base/SHA256SUMS" -O "$base/SHA256SUMS.asc" -O "$base/IMAGE_DIGEST" \
  -O "$base/$tgz"
cat ~/telcoin-node-scripts/lib/release-keys/*.asc | gpg --dearmor > release-keys.gpg
gpgv --status-fd 1 --keyring ./release-keys.gpg SHA256SUMS.asc SHA256SUMS
git clone --quiet https://github.com/Telcoin-Association/telcoin-network.git tn-main
git -C tn-main archive "refs/tags/$TAG" chain-configs | tar -xf -
sha256sum --check --ignore-missing SHA256SUMS
tar -xzf "$tgz"
telcoin-network-$TAG-x86_64-unknown-linux-gnu/telcoin-network --version
git -C tn-main rev-parse "refs/tags/$TAG^{commit}"
```

Go on only if all of these hold:

- `gpgv` prints a `GOODSIG` line and no `BADSIG`, `EXPSIG`, `EXPKEYSIG`, `REVKEYSIG` or `ERRSIG` line. Read those status lines rather than the summary, because `gpgv` also prints `Good signature from`, and exits 0, when the signing key has expired or been revoked.
- `sha256sum` prints `OK` for all five files `SHA256SUMS` lists: the tarball, `IMAGE_DIGEST`, and the chain configs `chain-configs/testnet/genesis.yaml`, `committee.yaml` and `parameters.yaml` (`chain-configs/mainnet/` for a mainnet tag). The `git archive` line puts the tagged chain configs at those paths; without it, `--ignore-missing` skips them without a word.
- `--version` prints `telcoin-network-cli Version: X.Y.Z` with the tag's version, a `Commit SHA:` equal to the commit the last line prints, and `Build Features: adiri` (possibly with `faucet`) on an `-adiri` tag, empty on a mainnet tag.

Then stop the node, install the new binary over that file, and start the node again:

```
sudo systemctl stop telcoin
sudo install -m 0755 <new binary> <binary path>
sudo systemctl start telcoin
```

On a committee node, run only the `install` line, which replaces the file while the node runs, then restart with `edit-config.sh` menu item `12) Restart node`, which runs the quorum check and waits for the epoch boundary. A signed-release install (`--install-method release`) does all of this for you. After any update, run `sudo bash ~/telcoin-node-scripts/check-node.sh`.

## 5.3 One-way updates

Two updates cannot be undone by putting the old binary or image back:

- An update from a release older than `v0.16.0-adiri` to `v0.16.0-adiri` or newer. The new release migrates the consensus store, the epoch packs under `consensus-db/epochs`, on its first start, and older releases cannot open the data directory afterwards.

- An update from `v0.16.0-adiri` to the next release or a later one. From the next release the node keeps its consensus database under `consensus-db/tndb/` (see Where things land (page 17)). Its first start copies the database over from `consensus-db/epoch`, `kad` and `cache`, then deletes the old files. `v0.16.0-adiri` is not refused afterwards, but it starts with empty epoch tables, with no record of the votes and proposals the node has already made, so on a committee node it can sign conflicting messages. Never run `v0.16.0-adiri` again on a committee node that has made this switch unless you put the snapshot back first.

Before either update, snapshot the data directory with the node stopped, whether or not the script asks about a snapshot. Prepare first (`1) Prepare only`), so the node is down only for the copy:

```
sudo systemctl stop telcoin
sudo cp -a /var/lib/telcoin /var/lib/telcoin.pre-update  # or take a disk snapshot instead
sudo systemctl start telcoin
```

The copy needs as much free space as the data directory; a disk snapshot from your cloud provider needs none on the server. On a committee node, stop it well away from an epoch boundary; `check-node.sh` shows the time to the next one.

For the update to `v0.16.0-adiri`, run `update-node.sh` again, choose `1) Apply now`, and answer yes when it asks about the snapshot. The first start can take minutes while the node migrates each epoch it opens; `journalctl -u telcoin -f` shows the progress. If the health check fails, the script leaves the node on the new release and does not stop it. A node that is still migrating comes up without help, and a node that stopped says why in the journal, so read the journal before you decide to go back. A damaged row in the epoch database stops `v0.16.0-adiri` at every start with a panic `Invalid bytes!` while it opens the database; see Service and keys (page 69). To go back, stop the node, move the migrated data directory aside and the snapshot back into its place (for the copy above, `/var/lib/telcoin.pre-update` back to `/var/lib/telcoin`), run the copy command from the error message, which puts the old start script back (with the old binary, unit and `node.env` on a release or source install), and start the node. An older release cannot start on the migrated data directory; it stops with `invalid version (should be 0)`.

For the update to the next release, the first start logs `copied <n> rows of the consensus DB from MDBX to tndb` once it has moved the database. A start that fails before then stops with `Cannot migrate the consensus DB from MDBX` in the journal; the old database stays untouched until the copy is complete, so the next start copies it again. When that line goes on with `Cannot check database (epoch)` and names `consensus-db/epoch`, the old database is still in place: move that directory aside, keep `consensus-db/epochs` in place, and start the node again once the network has closed that epoch (see Service and keys (page 69)). On a committee node, apply this update with `update-node.sh` from a terminal, without `TN_ASSUME_YES`, not from the Node Manager UI: the UI's apply rolls back on its own after a failed check, which would start `v0.16.0-adiri` with empty epoch tables. If a check fails after this update and the script asks `Roll back to previous binary?` or `Roll back to previous image (<image>)?`, answer no, and email devs@telcoin.org before you go back: only the snapshot undoes this update.

## 5.4 Restarts and the epoch boundary

A node that votes in the committee should not be down when the epoch changes: the epoch closes at the first commit at or after the boundary, and the committee exchanges epoch votes for 60 to 75 seconds afterwards. So these scripts check before they restart the node: `update-node.sh` when it applies an

update, `edit-config.sh`, `install-caddy.sh` (`rpc-enable` and `rpc-disable`), `setup-observability.sh` and `prepare-stake.sh --rotate-address`. In the Node Manager UI the Stop and Restart buttons, the tracing toggle and saving the advertised node name follow the same rule: the page asks first, then the Node Manager takes the update lock, runs the quorum check, waits for the epoch boundary when the node votes, checks again and only then stops the node. A plain `systemctl` restart and the Start button act at once.

Before they stop a node that votes in the committee, `update-node.sh`, `edit-config.sh`, `install-caddy.sh`, `setup-observability.sh` and a `setup-node.sh` re-run over a running node also count the committee members that are already down. If stopping this node as well would halt consensus, the script refuses and changes nothing on the running node: `... Refused: wait until the others are back, or, for a coordinated restart, run again with --allow-quorum-risk.` When the members cannot be counted, testnet and devnet go ahead with a warning, and mainnet refuses for a staked node. `--allow-quorum-risk` (`TN_ALLOW_QUORUM_RISK=1` for `install-caddy.sh` and `setup-observability.sh`) turns the refusal into a warning; use it only for a restart you have coordinated with the other operators.

When the node reports `CvvActive` and the boundary is within the margin, the script waits until the epoch closes, then for the settle time while the new committee forms, never longer than 30 minutes in all. Both follow the epoch length, which the script reads together with the boundary:

- margin: the epoch length divided by 72, held between 300 and 900 seconds;
- settle time: the epoch length divided by 240, held between 90 and 300 seconds.

On testnet's six-hour epochs (21600 seconds) that gives five minutes and 90 seconds; a seven-day epoch gives 900 and 300 seconds. The wait's lines end with the values and their source, for example `Margin 300s and settle 90s, derived from a 21600 s epoch.` It prints a line that starts `Waiting for epoch <N> to close before restarting`, with the time to the boundary and the limit, then a progress line every 15 seconds. Nodes outside the committee, staked validators between seats included, are not delayed, and a restart that rolls back a failed change never waits.

To skip the wait, pass `--no-epoch-wait` (`update-node.sh` and `edit-config.sh`) or set `TN_SKIP_EPOCH_WAIT=1` through `sudo`, for example `sudo TN_SKIP_EPOCH_WAIT=1 bash ~/telcoin-node-scripts/update-node.sh`. On a committee node, check the time to the next boundary with `check-node.sh` before a restart that does not wait (see The epoch section (page 25)), or restart with `edit-config.sh` menu item `12) Restart node`, which waits.

If a committee node is stopped at the epoch boundary anyway, it repairs its epoch records when it starts again and logs warnings that need no action (see Service and keys (page 69)). It still misses the epoch votes while it is down, so keep to the wait.

These environment variables tune the wait:

| Variable | Default | Meaning |
|---|---|---|
| `TN_SKIP_EPOCH_WAIT` | unset | `1` skips the wait. |
| `TN_EPOCH_MARGIN` | Epoch length / 72, 300 to 900 (300 on testnet) | Seconds before the boundary inside which a restart waits. A whole number you set is used as given. |
| `TN_EPOCH_SETTLE` | Epoch length / 240, 90 to 300 (90 on testnet) | Seconds to wait after the epoch changes. A whole number you set is used as given. |
| `TN_EPOCH_JITTER_MAX` | 30 | Then a random pause of up to this many seconds, so nodes that waited for the same boundary do not restart together. `0` turns it off. The committee check runs again after the wait. |
| `TN_EPOCH_WAIT_MAX` | 1800 | Cap on the whole wait, settling included. `0` turns the wait off. A cap you set is kept as set; when it is shorter than the margin, the settle time and two progress intervals together (420 seconds on testnet with the defaults), the script warns that the restart may come before the epoch has settled and suggests a larger cap or `TN_SKIP_EPOCH_WAIT=1`. |
| `TN_EPOCH_POLL` | 15 | Seconds between progress lines, held between 5 and 20. |

The Node Manager UI runs the same scripts, so an update or a config save there can wait too (see Node Manager UI (page 36)).

## 5.5 Change the configuration

```
sudo bash ~/telcoin-node-scripts/edit-config.sh
```

| Menu item | What it changes |
|---|---|
| `1) Listener addresses` | `PRIMARY_LISTENER_MULTIADDR` and `WORKER_LISTENER_MULTIADDR`. |
| `2) Metrics address` | The `--metrics` address; `off` removes the flag. |
| `3) Log verbosity` | `-v` to `-vvvvv`. |
| `4) RPC access` | `1) Private (recommended)` keeps reth on `127.0.0.1`; `2) Public` binds it to `0.0.0.0`, which mainnet refuses. Not for public RPC (see the cautions below). |
| `5) BLS passphrase` | `1) Change the stored passphrase`, or `2) Seal the credential again (TPM or PCR change)`. It never re-encrypts the keys. On a release whose keytool has `verify`, it checks the passphrase against the key first (below). |
| `6) P2P ports` | The P2P listener ports. |
| `7) Docker image` | The image a Docker install runs, as a release tag (resolved to its digest) or a digest. On mainnet, a release tag. |
| `8) Bootstrap peers` | A peers map in place of the genesis list (see Optional node flags (page 23)), or none. |
| `9) State export` | Off, every epoch's export, or the last N epochs. |
| `10) Private forward targets` | `allow_private_forward_targets` in `parameters.yaml`; private networks only. |
| `11) Refresh chain configs` | Pulls the latest genesis, committee and parameters and checks them against the pinned hashes. |
| `12) Restart node` | |
| `13) Exit` | |

Before the first change to a file, it copies the file into `/var/backups/telcoin/` (root only) and prints the copy's path and SHA-256. After an edit it asks `Restart the node now to apply changes?`; No keeps the change for the next restart.

For a single edit without the menu, use `--set`, with or without `--json`:

```
sudo bash ~/telcoin-node-scripts/edit-config.sh --set state_export=3
```

`--set` restarts the node and checks that it stays up; if it does not, it puts back every file the edit changed and restarts the node on the old configuration. A run stopped before the restart, during the epoch wait for example, puts the files back too. When the value is already in place, nothing is written and the node is not restarted.

| Field | Values |
|---|---|
| `primary_listener`, `worker_listener` | A multiaddr: `/ip4/<addr>/udp/<port>/quic-v1` or `/ip6/...`. The UDP port must match the external address the node advertises. |
| `metrics` | `<IPv4>:<port>` adds or changes `--metrics`; `off` removes it. The address is `127.0.0.1` unless you pass `--allow-public-metrics`, which `--json` runs do not take. |
| `verbosity` | `-v`, `-vv`, `-vvv`, `-vvvv` or `-vvvvv`; `--json` runs stop at `-vvv`. |
| `docker_image` | A release tag of the node's network, resolved to its digest, or `<repository>@sha256:<digest>` from the network's repository. `latest` and other repositories are refused, and so is a release older than the network's oldest supported one (`v0.13.0-adiri` on testnet). On mainnet a digest alone is taken only for a mainnet release the node has installed, so give the release tag. Docker installs only. The image is pulled first. |
| `bootstrap_peers` | An absolute path to a peers map, checked as at setup and installed as `/etc/telcoin/bootstrap-peers.yaml`, or `none` to remove the flag and the file. Needs `v0.15.0-adiri` or later. |
| `state_export` | `off`, `unlimited` (every export kept; `v0.13.0-adiri` or later) or a number of epochs to keep, 1 to 999999 (`v0.15.0-adiri` or later). |
| `allow_private_forward_targets` | `true` or `false`. `true` lets the node forward transactions to committee RPC endpoints on private addresses. `edit-config.sh` refuses `true` on testnet and mainnet, so it is for private networks such as a devnet. |

Each node flag is checked against the installed release before anything changes. Every edit takes the update lock, so it is refused while `update-node.sh` applies an update: `an update is in progress (PID N); try again when it has finished`. Before it restarts a node that votes in the current committee, `edit-config.sh` runs the quorum check and waits for the epoch boundary (see Restarts and the epoch boundary (page 31)); `--no-epoch-wait` skips the wait, and `--allow-quorum-risk` overrides a refusal for a coordinated restart. If the quorum check refuses, the edit is put back.

`sudo bash ~/telcoin-node-scripts/edit-config.sh --reseal` seals the passphrase credential again with the server's current TPM and PCR state, for example after a firmware update changed a PCR it is bound to. Add `--bind-pcrs <list>|none` to change the PCR binding, or `--allow-no-tpm` on mainnet to seal it with the host key alone. When the old credential no longer decrypts on the server, it asks you to type the passphrase. It restarts the node the same way an edit does and puts the old credential back if the node cannot unlock its key.

Before `5) BLS passphrase`, `--reseal` or `--migrate-credential` writes anything, `edit-config.sh` asks the node release whether its keytool has a `verify` command. When it has, the script runs `keytool verify` as the service account, with the passphrase on standard input: the new passphrase for a change, the one being moved for `--migrate-credential`, and for a reseal the one the credential holds, or the one you type when it no longer decrypts. A passphrase that does not open the BLS key stops the action with `... does not open the BLS key in <data dir> (keytool verify, exit N); nothing was changed. Use the`

`passphrase the key was created with.` On a release without the command, the script logs `keytool verify is not in this release; skipped` and goes on as before, so there the passphrase is stored without a check against the key.

`11) Refresh chain configs` takes the network from `.node-meta`, refuses files that do not match the SHA-256 pins that ship with the scripts (new pins arrive with a signed scripts release), and refuses unless the node's genesis and the new one declare the same chain ID. On a source build checked out at a release tag, `git pull` fails and the refresh stops with `git pull failed in /opt/telcoin-source. Chain configs unchanged.`

Two cautions:

- For public RPC, use `install-caddy.sh` (see Public RPC (https and wss) (page 42)), not the menu's `4) RPC access` item. reth must stay on `127.0.0.1`.
- Change the P2P ports only to fix a conflict. The new ports must be open in every firewall layer and must match the multiaddrs in `node-info.yaml`.

The README covers editing the configuration in more detail.

## 5.6 Service commands

```
sudo systemctl start telcoin
sudo systemctl stop telcoin
sudo systemctl restart telcoin
journalctl -u telcoin -f
sudo tail -f /var/log/telcoin/telcoin.log   # when the node logs to files
sudo systemctl reset-failed telcoin   # after too many failed restarts
```

A restart with `systemctl` does not wait for the epoch boundary (see Restarts and the epoch boundary (page 31)). The README has the full command reference.

## 5.7 Node Manager UI

The optional web UI shows health, logs and configuration, and can run updates. Install it on the node, from your own `~/telcoin-node-scripts`:

```
sudo bash ~/telcoin-node-scripts/ui/install-ui.sh
```

The installer refuses a checkout that anyone other than root, you or another administrator (a member of `sudo` or `adm`) can write to. It validates the sudo whitelist for the UI's privileged helper with `visudo` before it changes anything that is running. If a Python package cannot be installed, a UI source file is missing, the helper does not parse or `visudo` rejects the whitelist, it stops there and the running UI stays as it was.

The UI listens on `127.0.0.1:8080` only, and its actions need an access token, which the installer writes to `/etc/telcoin-ui/token`. Reach it from your own machine through an SSH tunnel. The helper script in a local copy of the deployment repository reads the token over SSH, opens the tunnel and opens `http://127.0.0.1:8080/#t=<token>` in your browser; Ctrl+C closes the tunnel:

```
./open-ui.sh user@SERVER_IP
```

By hand, open the tunnel, read the token on the server, and open `http://127.0.0.1:8080/#t=<token>` yourself:

```
ssh -L 8080:127.0.0.1:8080 user@SERVER_IP
sudo cat /etc/telcoin-ui/token   # on the server, in that session
```

`sudo bash ~/telcoin-node-scripts/ui/install-ui.sh --rotate-token` replaces the token; links that hold the old one stop working.

To reach the dashboard on a public HTTPS name, run `sudo bash ~/telcoin-node-scripts/install-caddy.sh` and choose `[1] Dashboard`. Caddy then serves it behind a login whose password needs at least 16 characters. Public access is read-only; management stays on the tunnel. The dashboard needs its own hostname, such as `dashboard.<your-node-domain>`, separate from the public RPC name. Both names must be DNS names with two or more labels, not IP addresses. On mainnet, `install-caddy.sh` does not serve the dashboard from a staked validator (see Public RPC (https and wss) (page 42)). See the README's web UI and external dashboard access sections.

`update-scripts.sh` does not redeploy the UI. When it brings new UI files, run the command it prints:

```
sudo bash ~/telcoin-node-scripts/ui/install-ui.sh --update
```

Until you do, the UI may show `The privileged helper is outdated.`: the helper is older than the page (this release needs helper API 3), and updates, config saves and public access fail.

What to expect from the UI:

- The setup wizard can make the node public at install time. In the Configure step, choose the `Public` card under RPC access and give the node's hostname; the field suggests the node's own name, and `Check DNS` tests it. A DNS mismatch is only a warning, because setup checks DNS again and keeps RPC private until the name resolves (see New install (page 42)). The review step reads `Public at <hostname>` or `Private`, and the completion card then shows whether public RPC is on and what `node-info.yaml` advertises.
- The validator view follows the chain. The UI asks the network's public RPC for the node's `getValidator` record, then the synced node, then its last saved answer, so a newly staked validator gets the validator view while it is still syncing. A banner says when the view comes from a cached answer, or when the role is unknown and the view defaults to observer.
- The setup wizard installs a signed release, a source build of a release tag, or Docker. The tag field starts at the latest release tag; the Docker field takes a release tag or an image digest and refuses `latest`. The Mainnet card stays disabled until the mainnet release.
- The Config tab has no fields for bootstrap peers, state export or private forward targets yet; set those with `edit-config.sh` (see Change the configuration (page 34)).

- The Config tab can re-seal the passphrase credential (`Re-seal the passphrase credential`, the same as `edit-config.sh --reseal`), for example after a firmware update changed the TPM state. It runs the quorum check and the epoch wait before it restarts the node.
- The System tab shows the Host Security Audit (`check-node.sh --audit`). When the firewall is off, its card asks for the SSH sources (addresses or CIDRs, /8 or narrower, at most 16) and enables the firewall with them as `--ssh-from`; on mainnet an empty list is refused.
- The same enable form has an optional `P2P sources` field, which takes the same kind of list and passes it as `--p2p-from`. When you fill it in, the P2P ports accept only those sources, so peers outside the list cannot reach the node. Left empty, the enable keeps the list the node already has, or opens the P2P ports to every address with the new-flow limit (see Firewall (page 20)). The field is in the enable form only, which shows while the firewall is off: to set, change or drop a P2P list once the firewall is on, run `firewall-setup.sh --enable --p2p-from <list>` or `--p2p-anywhere` on the server, and use `firewall-setup.sh` for `--ssh-anywhere` too. The firewall card shows the kept list as `P2P sources` and the limit's state as `P2P new-flow limit`.
- The Network tab links the network's public status page: on testnet, `https://status.adiri.telcoin.network/`. On mainnet and devnet it shows no status link.
- Before public RPC is enabled, the page shows the same exposure summary `install-caddy.sh` prints and waits for you to confirm.
- The update panel reports a refused downgrade, a node that is not voting after an update, and a release tag that now points at another commit.
- When the copy of the scripts the UI runs is older than the UI needs, an action stops with a message to run `update-scripts.sh` and then `install-ui.sh --update`.
- Updates and config saves run the same scripts, so they wait for the epoch boundary too (see Restarts and the epoch boundary (page 31)). A progress pane can show `Waiting for epoch N to close` for up to 30 minutes, with a line every 15 seconds. Keep the tab open until the result line appears, because leaving the tab stops the script. Leaving an update during the wait cancels it before the node is stopped, and the prepared update stays for a later apply; leaving it once the apply has started can cut the apply off part way. A config save that is stopped before it restarts the node is rolled back: every file it changed is put back as it was.
- The one-way update to `v0.16.0-adiri` (see One-way updates (page 30)) shows its warning in the progress pane when you prepare it. Apply does not ask about a snapshot, so take it before you press Apply, and expect the health check to take up to 10 minutes after the restart. The one-way update to the next release shows no warning here, and its apply rolls back on its own after a failed check, so a committee node takes that update from a terminal (see Update the node (page 28)).

## 5.8 Testnet add-ons

You can turn on the health monitor, log shipping, metrics shipping and VPN admin SSH after setup:

```
sudo bash ~/telcoin-node-scripts/setup-observability.sh   # health, logs, metrics
sudo bash ~/telcoin-node-scripts/setup-vpn.sh             # VPN admin SSH
```

The add-ons are for testnet only. Mainnet refuses them, and a mainnet install refuses to run on a server where one is present.

Turning on the health monitor, log shipping or metrics shipping adds flags to the node command (`--healthcheck`, `--log.file.format json`, `--metrics`) and restarts the node. Log shipping has the node write its own log files under `/var/log/telcoin/node/`. On a committee node the restart runs the quorum check and waits for the epoch boundary (see Restarts and the epoch boundary (page 31)). If a flag cannot be added, for example because the node command ends in a `#` comment, the enable fails without restarting the node, names the flags to add by hand, and does not record the add-on as on. Add the flags and run it again. The health monitor also needs `ufw` active (see Firewall (page 20)).

Log and metrics shipping share one ingest token, which you get from the Association. The token belongs to the Association's hub, not to your node: the hub checks one shared token, so when the Association replaces it, rotate it on each node that ships. Setup keeps it in its own file, `/etc/telcoin/alloy/ingest-token` (root, group `telcoin-alloy`, mode 0640), which Alloy reads as a file; it is never in Alloy's environment, its unit or its command line. To replace it, pass the new token on standard input:

```
read -rs TOKEN   # paste the new ingest token; nothing is shown
printf '%s\n' "$TOKEN" | sudo bash ~/telcoin-node-scripts/setup-observability.sh \
  --rotate-token --token-stdin
unset TOKEN
```

The rotation writes the token file, restarts Alloy alone (the node keeps running), and checks that Alloy stays up; if it does not, the old token and files go back and the run exits 1. Run from a terminal without `--token-stdin`, it asks for the token without showing it, and the menu has the same action as `8) Rotate the ingest token (Alloy restarts, the node does not)`. A wrong token keeps Alloy running while the hub refuses every push, so check the shipping counters the run prints afterwards.

A server set up before `scripts-v2.1.0` keeps the token in `/etc/telcoin/alloy/telcoin-alloy.env` until the next enable, partial disable or rotation moves it into the token file, which prints `/etc/telcoin/alloy/telcoin-alloy.env no longer holds the ingest token: ...`. Until then `7) Show status` in `setup-observability.sh` and `check-node.sh` warn, and a rotation with the same token is enough to move it. Turning off both log and metrics shipping shreds the token file, so turning either on again asks for the token.

The health monitor gives the node the port alone, `--healthcheck 43174`. On `v0.16.0-adiri` the node then listens on every interface, and the `ufw` rule limits the port to the Association monitor. From the next release, given the port alone, it listens on `127.0.0.1` only. The scripts' own status checks still reach it, because they probe `127.0.0.1`, but the Association monitor and every other host outside the server cannot, whatever `ufw` allows. A monitor outside the server then needs the flag with an address: `--healthcheck 0.0.0.0:43174` to listen on every interface (keep the `ufw` rule that limits the port to the monitor), or `--healthcheck <address>:43174` with an address the server owns, such as the private interface address a cloud server's public IP maps to. `v0.16.0-adiri` refuses the address form, so the form has to match the release the node runs. From the next release a bind that fails, because the port is in use or the address is not on this server, stops the node at start with the system's error, and a bind on every interface logs a warning that begins `listener bound on every interface`.

Read the testnet add-ons guide for what each one shares and how to turn it off. Read the WireGuard VPN guide before you enrol in the VPN.

## 5.9 Key lifecycle

The node's identity is `node-keys/` and `node-info.yaml` in the data directory. The BLS key in `node-keys/` is wrapped with your BLS passphrase, so the key files and the passphrase together are the key in the clear. The stored passphrase, `/etc/telcoin/bls-passphrase.cred`, is not part of a backup: it is sealed to the server that wrote it (its host key, plus the TPM with `host+tpm2`) and decrypts nowhere else.

- Backup. Copy `node-keys/` and `node-info.yaml` off the server as in Back up the keys now (page 19), and keep the passphrase offline, apart from them. A backup must not sit on the node's own disk, or in a snapshot or image of it. No script makes the backup for you.
- Snapshots are copies. A disk snapshot or image of the node holds the keys and the stored passphrase, and so does a copy of the data directory such as `/var/lib/telcoin.pre-update` (see One-way updates (page 30)). With `CREDENTIAL_KEY=host` the host key that decrypts the credential, `/var/lib/systemd/credential.secret`, is on the same disk, so the snapshot alone can start the validator on another machine. Keep every snapshot and image as safe as the backup, never boot a machine from one while the node runs, and delete them all when you retire the node.
- Restore, only once the old server is gone and cannot start again. Two servers that run one BLS key sign conflicting votes, and no script detects a second copy. The steps follow this list.
- Rotation. The BLS key cannot be rotated: new keys are a new validator, with a new ConsensusNFT and a new execution address (see What not to do (page 53)). Before you stake, `prepare-stake.sh --rotate-address` moves the keys to another execution address (see Change the execution address before you stake (page 49)). `edit-config.sh 5) BLS passphrase` changes only the stored copy of the passphrase and never re-encrypts the key, so it takes only the passphrase the key was created with; on a release whose keytool has `verify`, it refuses one that does not open the key (see Change the configuration (page 34)).
- Decommissioning. `remove-node.sh` shreds the keys, `/etc/telcoin` and `/var/backups/telcoin`, and says itself that this is not enough: shred is best effort on SSDs, copy-on-write and cloud disks, and `Decommissioning a host that held a validator key means destroying or crypto-erasing its disk and deleting every snapshot and image taken from it; shred alone does not do that.` Do both. Leaving the validator set is a separate, on-chain step (see Rewards and exit (page 52)).
- Lost passphrase. Without the passphrase the wrapped key cannot be opened, so the key is lost. While the server that sealed the credential still decrypts it, root can read the passphrase back there with `sudo systemd-creds decrypt --name=bls-passphrase /etc/telcoin/bls-passphrase.cred -`. That prints the passphrase, so run it only on that server, in a private terminal, and store the result offline. Otherwise, before you stake, generate new keys; after you stake, exit with the execution wallet (`beginExit()`, then `unstake`; neither needs the BLS key) and start again as a new validator after talking to the Association.

To restore on a new server, create the service account and the data directory before setup, put the backup into the data directory as that account's, and write the passphrase into the root-only file setup reads. For the archive made in Back up the keys now (page 19), copied into your home directory on the new server, with the default account and paths:

```
sudo groupadd --system telcoin
sudo useradd --system --no-create-home --shell /bin/false --gid telcoin telcoin
sudo install -d -o telcoin -g telcoin -m 0700 /var/lib/telcoin
sudo tar -C /var/lib/telcoin -xzf ~/telcoin-keys.tgz
sudo chown -R telcoin:telcoin /var/lib/telcoin/node-keys /var/lib/telcoin/node-info.yaml
sudo install -d -m 0755 /etc/telcoin
read -rs PASS   # type the BLS passphrase; nothing is shown
printf '%s\n' "$PASS" | sudo sh -c 'umask 077; cat > /etc/telcoin/bls-passphrase'
unset PASS
sudo bash ~/telcoin-node-scripts/setup-node.sh
```

Answer No to `Overwrite existing keys?`. Setup keeps the keys and, on a server with encrypted credentials, moves the passphrase file into a new credential sealed on this server and shreds the file. Do not copy the old server's `.cred`: setup keeps a credential it finds without decrypting it. A `--json` setup refuses existing keys, so a restore runs interactively. Setup does not check the passphrase against the key; a wrong one shows when the node cannot decrypt its key at start (see Service and keys (page 69)). Delete `~/telcoin-keys.tgz` from the new server with `shred -u` once the node runs.

## 5.10 Remove a node

If the node serves public RPC, turn that off first, because `remove-node.sh` does not touch Caddy:

```
sudo bash ~/telcoin-node-scripts/install-caddy.sh --phase=rpc-disable
sudo bash ~/telcoin-node-scripts/remove-node.sh
```

The removal script takes the update lock; while another script holds it, the removal stops before it changes anything. It asks before each component: service, Docker container and image, chain data, keys, binary, source tree, logs, service user, and the UI. Deleting the keys needs you to type `DELETE`. The keys, the passphrase credential and the rest of the configuration are shredded, not just deleted. Do not delete the keys of a staked validator unless you hold a tested backup. Testnet add-ons are removed without a question. The host baseline and the P2P new-flow limit go with the last node on the server; while another node unit remains, both stay for it. If you keep the service user, it asks before it removes the `telcoin-log` group. A removal that stopped part way can be finished: run the script again, and it lists the files left behind and asks before it removes them. Menu option `2) Wipe chain data only (keeps keys and config)` keeps the keys and configuration and makes the node resync from scratch.

# 6 Public RPC (https and wss)

Public RPC serves your node's JSON-RPC at `https://<domain>/` and its WebSocket at `wss://<domain>/`. It also advertises both URLs in `node-info.yaml` so wallets and gateways can discover the node. reth itself stays on `127.0.0.1`. Caddy holds the TLS certificate on port 443 and proxies to reth on loopback.

Turning it on is a deliberate choice. The Caddy site has no authentication and no rate limiting, so anyone can send requests to it. It suits observers that are meant to serve the public.

> **IMPORTANT**
> Other nodes forward transactions to the RPC URLs the committee advertises. A validator therefore has to advertise a reachable RPC URL to receive forwarded transactions. Choose that URL deliberately and protect the endpoint behind it. The Caddy site has no rate limiting of its own, so put rate limits in front of it, keep reth on `127.0.0.1` behind Caddy, and allow nothing else through the firewall.

On mainnet, `install-caddy.sh` refuses public RPC and the dashboard on a staked validator, and on any node it cannot confirm is not one (no `VALIDATOR_ADDRESS`, an unreadable stake status or an unknown network). Its message suggests serving public RPC from a separate RPC node.

The README section on the public RPC endpoint covers the internals. Its public node tuning section lists the reth request limits and caches to review before you serve the public. The steps follow.

## 6.1 DNS first

Create an A record for the node's public name that points at the server's inbound public IP. Wait until it resolves before you enable anything. Caddy requests a Let's Encrypt certificate as soon as the site loads. If the name does not point at the server yet, issuance fails and repeated attempts get rate-limited.

- Allow inbound TCP 443 (required) and TCP 80 (recommended) in every firewall layer, and forward both on the router if the server is behind NAT.
- Behind NAT, or on a server with several addresses, pass the inbound IP with `--public-ip <ip>`. The address the scripts detect is the outbound one, which can differ.
- The RPC name and the dashboard name must be different, for example `node7.example.com` for RPC and `dashboard.node7.example.com` for the dashboard.
- Caddy 2.8.0 or newer. `install-caddy.sh` installs Caddy from its apt repository when it is missing, and refuses an older one before it writes anything, with the install instructions.

## 6.2 New install

```
DOMAIN=node7.example.com   # your node's public name
sudo bash ~/telcoin-node-scripts/setup-node.sh --rpc-domain "$DOMAIN"
```

Behind NAT, run this instead, with the inbound public IP:

```
sudo bash ~/telcoin-node-scripts/setup-node.sh --rpc-domain "$DOMAIN" \
  --public-ip 203.0.113.10
```

Or answer `2) Public` at the RPC access prompt. With a domain, key generation writes `https://<domain>/` and `wss://<domain>/` into `node-info.yaml` when the release's keytool supports advertising RPC (`--rpc-http`, present since `v0.12.0-adiri`). After the node starts, setup checks DNS and runs `install-caddy.sh --phase=rpc-enable`. That run does not restart the node when `node-info.yaml` already holds those URLs. If DNS is not ready, setup finishes anyway and prints the command to run later.

`--no-public-rpc` keeps RPC private without asking. It cannot be combined with `--rpc-domain` or `--rpc-public`; setup stops with an error (an `error` event in `--json` mode). `--rpc-public` without `--rpc-domain` leaves RPC private and prints a warning with the command to enable it later.

## 6.3 Existing node

Check DNS first, then enable:

```
DOMAIN=node7.example.com   # your node's public name
sudo bash ~/telcoin-node-scripts/install-caddy.sh --phase=rpc-check-dns \
  --rpc-domain "$DOMAIN"
sudo bash ~/telcoin-node-scripts/install-caddy.sh --phase=rpc-enable \
  --rpc-domain "$DOMAIN"
```

`rpc-check-dns` changes nothing and exits 0 only when the name reaches this server. `rpc-enable` first prints what going public exposes (every JSON-RPC namespace the node serves, to anyone, with no rate limit at the edge) and asks `Expose this node's RPC publicly? [y/N]`. Outside a terminal it needs `--yes`; setup passes it for you when you gave `--rpc-domain`. `rpc-enable` writes the Caddy site and reloads Caddy, and adds `ufw` rules for 80 and 443, also while `ufw` is off, so they are in place when you turn it on. It makes sure reth serves WebSocket, and sets the advertised URLs in `node-info.yaml` (the first worker's entry) with the node's own `keytool set-rpc`. It restarts the node once, and only if `node-info.yaml` or the launch command changed; on a committee node it first runs the quorum check and waits for the epoch boundary (see Restarts and the epoch boundary (page 31)). It records `PUBLIC_RPC_DOMAIN`, `PUBLIC_RPC_URL` and `PUBLIC_WS_URL` in `.node-meta`. Add `--public-ip <ip>` behind NAT. If the dashboard already uses this name, create the A record for `dashboard.<domain>` and add `--move-dashboard-to dashboard.<domain>` to move it in the same run. For a guided run, start `sudo bash ~/telcoin-node-scripts/install-caddy.sh` and choose `[2] Public RPC endpoint`.

The site Caddy serves from `/etc/caddy/Caddyfile` proxies JSON-RPC requests to reth's HTTP port and WebSocket upgrades to its WebSocket port. The upgrade match ignores case, so a proxy that sends `Connection: upgrade` gets through. A browser that opens the URL gets HTTP 405 and a short page naming the endpoint, with a `curl` example. Request bodies above 2 MB (Caddy counts that as 2,000,000 bytes) get HTTP 413, and Caddy cuts the connection once the cap is passed, so reth never receives a whole oversized request. The cap covers JSON-RPC over HTTPS; WebSocket messages are not capped at the edge. Every change backs up the Caddyfile to `/etc/caddy/Caddyfile.bak.<time>`, and the newest five backups are kept.

## 6.4 Verify

```
curl -s https://$DOMAIN/ -H 'content-type: application/json' \
  -d '{"jsonrpc":"2.0","id":1,"method":"eth_chainId","params":[]}'
```

On testnet the answer is `"result":"0x7e1"` (chain ID 2017). The first request after enabling can take a few seconds while the certificate is issued.

For WebSocket, connect with `websocat wss://$DOMAIN/` or `wscat -c wss://$DOMAIN/` and send the same JSON request. Opening `https://$DOMAIN/` in a browser shows the 405 page, which confirms that Caddy serves the name.

```
sudo bash ~/telcoin-node-scripts/check-node.sh
sudo bash ~/telcoin-node-scripts/install-caddy.sh --phase=rpc-status
```

The health check has a public RPC block that probes https and wss through Caddy and shows what `node-info.yaml` advertises. It ends with `public RPC: OK`, or with `public RPC: WARN -- <reasons>` plus the command that fixes it. Run it with `sudo`; without root it can only report `public RPC: unknown (.node-meta not readable — run with sudo)`. `--phase=rpc-status` shows whether the site is enabled, what is advertised, and whether reth's WebSocket port is listening.

`check-node.sh` also warns when `--http.api` or `--ws.api` on the node command names `debug`, `trace` or `admin`, because Caddy serves those methods to anyone who reaches the name. Remove them from the launch file and restart the node.

## 6.5 Turn it off

```
sudo bash ~/telcoin-node-scripts/install-caddy.sh --phase=rpc-disable
```

This removes the Caddy site, clears the advertised URLs from `node-info.yaml` and restarts the node, after the epoch wait on a committee node. It also drops the `PUBLIC_RPC_*` lines from `.node-meta`, and removes the `ufw` rules for 80 and 443 once no Caddy site remains.

# 7 Validators: stake and activate

The stake and activation calls go to the ConsensusRegistry contract at `0x07E17e17E17e17E17e17E17E17E17e17e17E17e1`, the same address on testnet and devnet. The BLS keys never leave the node. On the node, `prepare-stake.sh` checks everything the stake depends on and prints the exact `cast send` commands; you run those commands on the machine that holds your wallet. The script never reads, asks for or prints a private key, and it sends nothing. The commands in this chapter match the deployed contracts.

The `cast` commands from Watch your status (page 49) on use these variables, set on the wallet machine:

```
REG=0x07E17e17E17e17E17e17E17E17E17e17e17E17e1
RPC=https://rpc.adiri.tel   # testnet; see Networks and endpoints
ADDR=0xYOUR_EXECUTION_ADDRESS
```

## 7.1 Prerequisites

- Association approval. Only GSMA-approved mobile network operators can validate; start with devs@telcoin.org.
- Hardware at the validator tier (see Hardware per role (page 7)).
- The node's P2P ports reachable from the internet (see Ports (page 9)).
- A synced node (see Confirm the node is syncing, then synced (page 25)).
- A publicly routable RPC URL advertised in `node-info.yaml` (see Public RPC (https and wss) (page 42)). Other nodes forward user transactions to the committee's advertised URLs and skip private addresses. Only a private network, such as a devnet, can lift that with `allow_private_forward_targets: true` in `parameters.yaml` (see Change the configuration (page 34)); `edit-config.sh` refuses `true` on testnet and mainnet.
- The execution address funded with the stake plus gas. The stake is 1,000,000 TEL on testnet today; `prepare-stake.sh` reads the current amount.
- Foundry's `cast` on the wallet machine, and the wallet holding your execution address.

## 7.2 Check the node with prepare-stake.sh

On the node:

```
sudo bash ~/telcoin-node-scripts/prepare-stake.sh
```

It prints each check as it runs it:

1. Network RPC. The RPC must serve this node's chain: testnet (2017) unless `NETWORK` in `.node-meta` names another network. The default is that network's public RPC (see Networks and endpoints (page 10)); `--network-rpc <URL>` picks another.
2. Validator address. The `execution_address` in `node-info.yaml`, shown with its EIP-55 checksum. It warns when `.node-meta` records a different `VALIDATOR_ADDRESS`.

3. Whitelist. Whether the address holds a ConsensusNFT. Without one the run stops: send the address to the Association for approval, and run the script again once the NFT is minted.
4. Validator status. An address that has not staked goes on. A staked one (status 1) skips to the `activate()` step. Any later status is reported, and the run ends with "Nothing to do".
5. Stake amount. What the registry asks for now, from `stakeConfig(getCurrentStakeVersion())`, in TEL and wei.
6. Balance. The address's TEL against the stake plus a gas allowance (the gas price times a million gas). If it is short, the script says how much to send and still prints the commands.
7. Stake calldata. Exported by the node's own keytool. No passphrase is needed.
8. Simulation. `stake()` is simulated from the address with `eth_estimateGas`. A predicted revert is named with what to do (see If prepare-stake.sh predicts a revert (page 48)), and no commands are printed.
9. Local node sync. A warning when the node trails the network by more than 50 blocks.

Then it prints the stake and activation commands, a note on signing, and the epoch arithmetic from When you get a committee seat (page 50). With `--json` the human output goes to stderr and one JSON object with every value the run read goes to stdout at the end.

The run ends with one line and an exit code:

| Exit | Last line | Next step |
|---|---|---|
| 0 | `Ready to stake: run the commands above.` | Send the stake (page 47). |
| 0 | `Staked: activate() is next.` | Activate (page 47) once the node has caught up. |
| 0 | `Nothing to do: ...`, because activation is under way, the validator is active, exiting or exited, or the address is retired | Watch your status (page 49). |
| 0 | `Rotated: node-info.yaml now names 0x....` | Run `prepare-stake.sh` again to export the calldata for the new address. |
| 1 | A usage or setup problem, such as `Run this as root: sudo bash prepare-stake.sh <args>` | Do what the message says. |
| 2 | The network RPC cannot be reached, serves another chain, or the on-chain state cannot be read; or keytool cannot export the stake calldata | For the RPC, check the server's internet access, or pass `--network-rpc <URL>` with an RPC for the node's chain. For keytool, act on its message, which the last line includes. |
| 3 | `Not ready: ...`, `Refused: ...` or `Cancelled: nothing changed.` | Fix what the lines above it name, then run it again. |
| 4 | `... node-info.yaml is back as it was (from <backup>).` | A rotation failed and was undone (see Change the execution address before you stake (page 49)). |

## 7.3 Send the stake

The script prints the stake command with your values filled in, for example:

```
cast send --rpc-url https://rpc.adiri.tel 0x07E17e17E17e17E17e17E17E17E17e17e17E17e1 \
  0x2fb0d025... \
  --value 100000000000000000000000 --from 0xYOUR_EXECUTION_ADDRESS --ledger
```

Copy it to the wallet machine and run it there. The calldata holds only public data: the node's BLS public key and its proof of possession.

`--ledger` signs on a Ledger; use `--trezor` for a Trezor. For a key held in software, use `--account <name>`, an encrypted Foundry keystore you create once with `cast wallet import <name> --interactive`, or `--interactive` to paste the key when `cast` asks for it. Never put the key itself on a command line: typed there it lands in your shell history, and passed with `--private-key` it shows in the process list to any user of that machine while `cast` runs. Prefer a hardware wallet for a funded validator address.

Wait for the transaction to be mined. Running `prepare-stake.sh` again then ends with `Staked: activate() is next.` and prints only the activation command.

## 7.4 Activate

Once the stake has landed (status 1, Staked) and the node has caught up, run the second command the script printed:

```
cast send --rpc-url https://rpc.adiri.tel 0x07E17e17E17e17E17e17E17E17E17e17e17E17e1 \
  'activate()' --from 0xYOUR_EXECUTION_ADDRESS --ledger
```

Status moves to `2` (PendingActivation) and then to `3` (Active) at the next epoch boundary. When you get a committee seat (page 50) says when the first seat can come.

## 7.5 If prepare-stake.sh predicts a revert

The simulation names the error the registry would return, and the run stops with exit 3 and `Not ready: stake() would revert with <Error>. Fix that before you send the stake.` When a `cast send` reverts, `cast` prints the error selector, and the error's name when it can decode it; the names are the ones below.

| Error | What it means | What to do |
|---|---|---|
| `InvalidProofOfPossession` | The proof of possession in `node-info.yaml` was signed for a different address than the one you stake from. The keys are fine. | Re-sign it for the staking address with `prepare-stake.sh --rotate-address` (see Change the execution address before you stake (page 49)). |
| `InvalidTokenId` | The address holds no ConsensusNFT, so it is not whitelisted. | Ask the Association for approval, then run the script again. |
| `RequiresConsensusNFT` | The ConsensusNFT for the address is not held by that address. | Email devs@telcoin.org. |
| `InvalidStakeAmount` | The stake configuration changed after the script read it. | Run the script again for the current amount. |
| `InvalidStatus` | The registry already holds a validator record for the address (the status number is shown), so it cannot stake again. | Check it with `sudo bash ~/telcoin-node-scripts/check-node.sh`. |
| `DuplicateBLSPubkey` | This BLS key is already registered. A BLS key can be staked only once. | If you staked it from another address, that address is this node's validator: check it with `check-node.sh --address <that address>`. Do not generate new keys. |
| `InvalidBLSPubkey` | The BLS public key in `node-info.yaml` is not a valid key, so the file was edited or damaged. | Restore `node-info.yaml` from your backup (see Back up the keys now (page 19)). |
| `EnforcedPause` | The registry is paused, so staking is closed for now. | Try again later. |
| `LowLevelCallFailure` | The registry could not run its BLS signature check. | Try again later. If it keeps failing, email devs@telcoin.org. |
| `Error` with a message | The registry rejected the call with that message. | Act on the message. |
| `Panic`, or an error the script does not know | An internal error in the registry, or error data the script cannot name. | Email devs@telcoin.org with the data the script prints. |

## 7.6 Change the execution address before you stake

The execution address is signed into the proof of possession in `node-info.yaml`. To stake from a different address than the one you entered at key generation, re-sign the proof of possession for it with the same keys:

```
sudo bash ~/telcoin-node-scripts/prepare-stake.sh --rotate-address 0xNEW_ADDRESS
```

Rotation works only while the current address has not staked and is not retired. The new address must not have staked or be retired either; a whitelisted address that has not staked, or one that is not whitelisted yet, is fine. Once an address has staked, the registry holds the BLS key for it and the script refuses with `Refused: 0x... has staked (...)`.

The script:

- Shows what will change and asks first. `--yes` skips the question, and a `--json` run needs it.
- Takes the update lock, so it refuses while `update-node.sh` is applying an update: `Refused: an update is in progress (PID N); try again when it has finished.`
- Copies `node-info.yaml` to `node-info.yaml.bak.<UTC time>` beside it.
- Runs `keytool generate pop` from the node's own release, which changes only `execution_address` and `proof_of_possession`, then checks that the BLS key and the node name are unchanged.
- Records the new address as `VALIDATOR_ADDRESS` in `.node-meta`.
- Restarts the node so it loads the new file, after the epoch wait on a committee node. `--no-restart` leaves the restart to you.

The script hands the passphrase to keytool on standard input. If keytool fails or its result is wrong, the script puts `node-info.yaml` back and exits 4; a wrong passphrase is the usual cause.

The passphrase comes from `TN_BLS_PASSPHRASE` first, then from `/etc/telcoin/bls-passphrase` when that file exists with mode 600, then from a prompt on a terminal. The script does not read the encrypted credential, and only a testnet or devnet node keeps the plain file: one below systemd 250, or one set up with `--passphrase-method stdin` (see How setup stores the passphrase (page 15)). So on most nodes, type the passphrase at the prompt, or pass the variable through `sudo`, which otherwise drops it:

```
read -rs TN_BLS_PASSPHRASE && export TN_BLS_PASSPHRASE
sudo --preserve-env=TN_BLS_PASSPHRASE bash ~/telcoin-node-scripts/prepare-stake.sh \
  --rotate-address 0xNEW_ADDRESS
```

After a rotation, run `sudo bash ~/telcoin-node-scripts/prepare-stake.sh` again; it exports the calldata for the new address.

## 7.7 Watch your status

```
cast call $REG \
  "getValidator(address)(address,uint32,uint32,uint8,bool,uint8,uint8)" \
  $ADDR --rpc-url $RPC
```

The seven lines are the validator address, activation epoch, exit epoch, status, `isRetired`, stake version and region. The fourth line is the status (see Status values (page 51)). The same information, with the next step spelled out, comes from:

```
sudo bash ~/telcoin-node-scripts/check-node.sh
```

`check-node.sh` finds the address itself, from `.node-meta` or the node; add `--address <0x...>` to check another one. Its epoch section (see The epoch section (page 25)) shows the current epoch and when it ends, whether the address is in the committee for this epoch and the next two, and a staked validator's activation epoch and earliest committee seat.

Other signals:

- The Node Manager UI shows the validator view as soon as the network reports status 1 to 4 for the node's address, even while the node is still syncing. On a synced node its Current Epoch tile counts down to the boundary and shows the activation epoch and the earliest seat.
- `tn_nodeMode` (see Checking by hand (page 26)) changes from `Observer` to `CvvActive` while the node holds a committee seat. A seated node that falls behind, for example during a long restart, reports `CvvInactive` until it has caught up, then logs `attempting to rejoin consensus` and votes again.
- The current epoch and its committee:

```
cast call $REG "getCurrentEpoch()(uint32)" --rpc-url $RPC
cast call $REG \
  "getCurrentEpochInfo()((address[],uint256,uint64,uint32,uint32,uint8))" \
  --rpc-url $RPC
```

The second call's output starts with the list of committee addresses.

## 7.8 When you get a committee seat

`activate()` mined in epoch E makes the validator active at epoch E+1. The committee for each epoch is fixed two epochs in advance, so the earliest seat is epoch E+3, the activation epoch plus 2. On testnet's six-hour epochs that is 12 to 18 hours after `activate()`, depending on how much of epoch E was left. `prepare-stake.sh` prints the live numbers, for example:

```
  The network is in epoch 581, which ends in about 2h 10m.
  activate() mined now: active at epoch 582, earliest seat at epoch 584.
```

Selection is random only when there are more eligible validators than seats. Between seats an Active validator reports `Observer`, which is normal.

## 7.9 Status values

| Status | Name | What it means | Your next step |
|---|---|---|---|
| (call reverts) | No record | No ConsensusNFT for this address. | Finish Association approval. |
| 0 | Undefined | NFT minted, not staked. | Run `prepare-stake.sh`, then stake (see Check the node with prepare-stake.sh (page 45)). |
| 1 | Staked | Stake received. | `activate()` once synced (see Activate (page 47)). |
| 2 | PendingActivation | Activation queued. | Wait for the next epoch boundary. |
| 3 | Active | Eligible for committee seats. | Keep the node healthy. |
| 4 | PendingExit | `beginExit()` called. | Wait until the protocol exits you. |
| 5 | Exited | Out of the validator set. | `unstake` from the epoch after the exit epoch; `check-node.sh` says when (see Rewards and exit (page 52)). |
| 6 | Any | A reserved sentinel. On its own you should never see it. | Email devs@telcoin.org. |
| 6 with `isRetired` true | Retired | `unstake` ran: NFT burned, stake returned. | None. This identity cannot stake again. |

`check-node.sh` prints `Status: Retired` for the last row and appends `(Retired)` to any other status that has `isRetired` set. The scripts treat statuses 1 to 4 as staked and everything else as not staked.

## 7.10 Stake by hand

`prepare-stake.sh` runs these steps for you. Use them to cross-check its output, or when you cannot run it.

Confirm the ConsensusNFT on the wallet machine; `1` means the Association has minted it, `0` means approval is not complete:

```
cast call $REG "balanceOf(address)(uint256)" $ADDR --rpc-url $RPC
```

Export the stake calldata on the node. The keytool reads only `node-info.yaml`, but it still wants a passphrase source, so pass `--bls-passphrase-source no-passphrase`; `-q` keeps its log line out of the output, so you get the calldata alone. Source builds (an `existing` install uses its own binary path):

```
sudo /opt/telcoin/telcoin-network -q --bls-passphrase-source no-passphrase \
  keytool export-staking-args --node-info /var/lib/telcoin/node-info.yaml --calldata
```

Docker installs:

```
IMAGE=$(sudo sed -n 's/^DOCKER_IMAGE=//p' /etc/telcoin/.node-meta)
sudo docker run --rm --user "$(id -u telcoin):$(id -g telcoin)" \
  -v /var/lib/telcoin:/home/nonroot/data:ro "$IMAGE" \
  telcoin -q --bls-passphrase-source no-passphrase \
  keytool export-staking-args --node-info /home/nonroot/data/node-info.yaml --calldata
```

Copy the `0x...` output to the wallet machine and check that it is calldata for `stake`:

```
CALLDATA=0x...   # paste the output here
[ "${CALLDATA:0:10}" = "$(cast sig 'stake(bytes,(bytes))')" ] && echo "calldata OK"
```

Read the stake amount for the current stake version:

```
VERSION=$(cast call $REG "getCurrentStakeVersion()(uint8)" --rpc-url $RPC)
cast call $REG "stakeConfig(uint8)(uint256,uint256,uint256,uint32)" $VERSION \
  --rpc-url $RPC
STAKE=$(cast call $REG "stakeConfig(uint8)(uint256,uint256,uint256,uint32)" \
  $VERSION --rpc-url $RPC | awk 'NR==1 {print $1}')
```

The four values are the stake amount in wei, the minimum reward withdrawal, the issuance per epoch, and the epoch length in seconds. `stake()` takes exactly the first value of the current epoch's stake version, so send that amount or the call reverts with `InvalidStakeAmount`. Do not read the amount from `getCurrentStakeConfig()`: it returns the newest configuration, which takes effect only at the next epoch. Read the amount just before you stake. Once you have staked, the amount that applies to your validator is fixed by the stake version recorded on it (the sixth field of `getValidator`).

Then stake, and once the stake has landed and the node is synced, activate (Send the stake (page 47) covers the signing options):

```
cast send $REG $CALLDATA --value $STAKE --from $ADDR --ledger --rpc-url $RPC
cast send $REG "activate()" --from $ADDR --ledger --rpc-url $RPC
```

## 7.11 Rewards and exit

Claim accrued rewards without exiting once they reach `minWithdrawAmount`, the second value of the stake configuration:

```
cast send $REG "claimStakeRewards(address)" $ADDR --from $ADDR --ledger --rpc-url $RPC
```

To leave, call `beginExit()` while Active:

```
cast send $REG "beginExit()" --from $ADDR --ledger --rpc-url $RPC
```

It reverts if too few validators would remain. The status moves to PendingExit (4), and the protocol exits you once no current or upcoming committee needs you. When the status reads Exited (5), wait until the epoch after the exit epoch (`check-node.sh` says when `unstake()` becomes eligible), then reclaim the stake:

```
cast send $REG "unstake(address,bool)" $ADDR false --from $ADDR --ledger --rpc-url $RPC
```

**WARNING**

`unstake` permanently retires the validator and burns its ConsensusNFT. Neither the address nor the BLS key can stake again, and validating again needs new keys and a new NFT.

The second argument, `acceptRewardShortfall`, stays `false`; `true` forfeits any rewards the Issuance contract cannot cover. `unstake` also works while the status is still Staked (1), before activation.

## 7.12 What not to do

- Never regenerate keys for a staked identity, and never pass `--force` to the keytool for one. The registered BLS key would no longer match the node, and the registry never releases a registered key, not even after `unstake`.
- Do not generate new keys to change the execution address. Before you stake, re-sign with `prepare-stake.sh --rotate-address` (see Change the execution address before you stake (page 49)); after you stake, the address cannot change.
- New validator keys are a new validator identity. Generate them in a new data directory, and only after talking to the Association.
- Never run the same keys on two servers at once.
- When you re-run `setup-node.sh` on a staked node, answer No to `Overwrite existing keys?`.

# 8 Automation

## 8.1 Setup without prompts

`setup-node.sh --json` runs in two phases so you can back up the keys in between. Pass the same flags to both phases. Keygen records the install method, the release, the Docker image or binary path and the node flags in `.node-meta`, and finalize reads them back for any of those you leave out. `vX.Y.Z-adiri` stands for the release you install (the newest final on the releases page, or a release candidate only when you mean one):

```
read -rs PASS   # type the BLS passphrase; nothing is shown
ARGS=(--network testnet --install-method docker --release-tag vX.Y.Z-adiri
  --address 0xYOUR_EXECUTION_ADDRESS
  --external-primary /ip4/203.0.113.10/udp/49590/quic-v1
  --external-worker  /ip4/203.0.113.10/udp/49594/quic-v1
  --listener-primary /ip4/10.0.0.5/udp/49590/quic-v1
  --listener-worker  /ip4/10.0.0.5/udp/49594/quic-v1)
printf '%s\n' "$PASS" | sudo bash ~/telcoin-node-scripts/setup-node.sh \
  --json --phase=keygen --passphrase-stdin "${ARGS[@]}"
unset PASS
```

Back up `node-keys/` and `node-info.yaml` now (see Back up the keys now (page 19)), then run the second phase in the same shell:

```
sudo bash ~/telcoin-node-scripts/setup-node.sh \
  --json --phase=finalize "${ARGS[@]}"
```

- `keygen` runs preflight, installs dependencies, builds, downloads or pulls the node, creates the service user, generates the keys and stores the passphrase. It writes no unit and starts nothing, and it refuses to run if `node-keys/` already exists.
- `finalize` writes the configuration, creates and starts the service, and enables public RPC when you pass `--rpc-domain`. It needs no passphrase.
- The passphrase comes from standard input with `--passphrase-stdin`, or from `TN_BLS_PASSPHRASE`; giving both is an error. Keep it off the command line.
- Either phase refuses to run once a `telcoin` unit exists; change an installed node with `edit-config.sh`, or remove it first.
- Each phase prints one JSON object per line on stdout, with `event` set to `step`, `log`, `error` or `done`. Every run ends with exactly one `{"event":"done","ok":true|false,...}`, and a failed run's `done` carries the reason in `msg`. Human-readable output goes to stderr.
- Finalize's `done` adds `operator_guide` (the URL of the operator runbook) and `public_rpc` (`enabled`, `domain`, `http`, `ws`).
- The inputs are checked before anything runs as root. A flag with no value, an unknown phase, network or install method, a malformed URL, a `--docker-image` that is not `<repository>@sha256:<digest>`, a relative `--binary-path`, and a `--release-tag` that is not a release tag of the network or is older than

`v0.13.0-adiri` each stop the run with an `error` event. A keygen run also stops there when `--address`, one of the four multiaddr flags or the passphrase is missing, or when a source or release install has no `--release-tag`.

- Finalize stops when a flag differs from what keygen recorded (install method, data directory, image or binary path); give the keygen value or leave the flag out.
- `--network` takes `testnet` (or `adiri`), `mainnet` or `devnet`. `--network mainnet` stops with `Mainnet is not enabled in this script release ...` until the scripts release that launches mainnet (see Networks and endpoints (page 10)).
- `--install-method` takes `source` or `release`, each with `--release-tag <tag>` (`--build-ref` is the older name), `docker` or `existing`. For Docker, `--release-tag` picks that release's image by digest and `--docker-image` takes a digest; give one or the other, not both. With neither, keygen picks the newest release image and records its digest.
- For `existing`, `--binary-path <path>` (absolute) names the binary and, in `--json` mode, implies `existing`. Without it, keygen searches PATH, `/usr/local/bin`, `/opt` and `/home`, and stops when it finds none.
- Node flags: `--bootstrap-peers <file>`, `--enable-state-export` and `--state-export-keep <n>` (see Optional node flags (page 23)).
- Passphrase and log flags: `--passphrase-method credential|stdin`, `--allow-no-tpm` and `--bind-pcrs <list>` (see How setup stores the passphrase (page 15)), and `--log-target journal|file` (see Logs (page 24)).
- Other flags: `--data-dir`, `--public-ip`, `--advertised-name`, `--service-user`, `--service-group`, `--genesis-dir`, `--enable-healthcheck-monitor`, `--host-baseline` (see Host baseline (optional) (page 22); keygen ignores it), and the public RPC flags from Public RPC (https and wss) (page 42).

## 8.2 Advertised RPC URL flags

`--rpc-domain <d>` fills in all four of these, so most installs never pass them:

| Flag | Written to | Derived from `--rpc-domain <d>` |
|---|---|---|
| `--rpc-http <url>` | `node-info.yaml`, published to the network | `https://<d>/` |
| `--rpc-ws <url>` | `node-info.yaml`; needs `--rpc-http` or `--rpc-domain`, otherwise it is ignored with a warning | `wss://<d>/` |
| `--public-rpc-url <url>` | `.node-meta PUBLIC_RPC_URL`, for the UI and tooling only | `https://<d>` |
| `--public-ws-url <url>` | `.node-meta PUBLIC_WS_URL` | `wss://<d>` |

An explicit flag wins and is used as given. `--rpc-http` and `--public-rpc-url` take an `http://` or `https://` URL, `--rpc-ws` and `--public-ws-url` a `ws://` or `wss://` one. Setup stops on a malformed URL, and warns when a URL or `--rpc-domain` points at a private address or name (loopback, RFC 1918, `.local`, `.internal`) that wallets on the internet cannot reach. An invalid `--public-ip` is ignored with

a warning. Derived URLs reach key generation only when the keytool supports `--rpc-http`; otherwise they are written when `rpc-enable` runs. If an explicit URL differs from the domain, setup warns that `rpc-enable` will replace it.

## 8.3 Other scripts

| Script | Non-interactive forms |
|---|---|
| `install-caddy.sh` | `--phase=rpc-status`, `rpc-check-dns`, `rpc-enable`, `rpc-disable` and the dashboard phases `status`, `check-dns`, `enable`, `disable`, with or without `--json`. Outside a terminal and without `--json`, `rpc-enable` needs `--yes`. The dashboard password comes on standard input with `--password-stdin` and needs at least 16 characters. |
| `update-node.sh` | `--json --check`, `--json --prepare --ref <tag>`, `--json --apply --yes [--no-epoch-wait] [--allow-quorum-risk]`, `--json --discard`. `--ref` takes a release tag of the network or, on a Docker install, the network's image by digest; it cannot start with `-`. A failed apply rolls back; its `done` line carries `"ok":false` and a boolean `rolled_back` (`true` when the rollback succeeded), and `"voting":false` when the node did not take part in consensus. An apply whose node was still catching up when the consensus check ran out of time ends `"ok":true` with `"voting":false` (see Update the node (page 28)). A one-way apply (see One-way updates (page 30)) never rolls back: its failed `done` carries `"rolled_back":false` and `"storage_migration":true`, and `--json --check` adds the same boolean, `true` when updating to `latest_ref` is one-way. A `--json --discard` after such an apply had started the new release keeps the launch files, sends a `warn` event, and its `done` carries `"storage_migration":true`. A refused downgrade's `done` carries `"downgrade":true`. |
| `firewall-setup.sh` | `--enable` or `--reset`, with `--ssh-from`, `--p2p-from`, `--ssh-anywhere` or `--p2p-anywhere`; `--json --status`, `--json --enable`, `--json --port <49590/udp\|49594/udp\|43174/tcp> <on\|off>`. `--json --status` adds `p2p_ports`, one object per P2P port with `port`, `proto`, `label` and `allowed` (`null` while `ufw` is off), and `p2p_new_flow_limit` with `active`, `rate`, `burst` and whether the limit comes back at boot (`persistent`). Exit 0 is done, 1 refused with nothing changed, 2 a step failed and the previous rules are back, 3 the put-back failed too (the message names the saved rule files). |
| `edit-config.sh` | `--set <field>=<value>`, with or without `--json`, for the fields in Change the configuration (page 34); `--reseal`; `--no-epoch-wait`, `--allow-quorum-risk`. |
| `check-node.sh` | `--audit` (see Check the host hardening (page 21)), `--network-rpc <URL>`, `--address <0x...>`. |
| `update-scripts.sh` | `--check` reports the installed and the newest scripts release without installing anything; exit 0, 1 or 2 (see Keep the scripts current (page 27)). |
| `prepare-stake.sh` | `--json`, `--network-rpc <URL>`, and `--rotate-address <0xNEW> --yes [--no-restart]`. With `--json` it prints one JSON object at the end; the exit codes are in Check the node with prepare-stake.sh (page 45). |
| `setup-observability.sh` | `--json` with `--enable-logs`, `--enable-metrics`, `--enable-health` or the matching `--disable-...`; `--token-stdin` takes the ingest token on standard input. `--rotate-token --token-stdin`, with or without `--json`, replaces the token and runs on its own (see Testnet add-ons (page 38)). Testnet only. |
| `remove-node.sh` | `--json --remove observer\|validator --scope service\|data\|keys --yes [--remove-ui]`. The `--remove` value is a UI slot name, not a role; either one removes the node. |

The status and DNS-check phases and `update-node.sh --json --check` print a single JSON object instead of an event stream. In `--json` runs, the epoch wait (see Restarts and the epoch boundary (page 31)) report as `step`, `log` and `warn` events. `TN_ASSUME_YES=true` answers yes to the scripts' yes/no prompts. `check-node.sh` has no JSON mode and, apart from `--audit`, always exits 0, so parse its verdict line.

# 9 Troubleshooting

Start with `sudo bash ~/telcoin-node-scripts/check-node.sh` and `journalctl -u telcoin -n 200`. The tables below group the common symptoms by area.

## 9.1 Sync and node mode

| Symptom | Likely cause | Fix |
|---|---|---|
| `tn_nodeMode` returns `Observer` while `getValidator` shows Active | Not seated yet, or between seats. | Normal. See When you get a committee seat (page 50). |
| `tn_nodeMode` stays at `CvvInactive` | A committee member that has not caught up. It votes again once it is within 20 rounds of the network's newest commit in the same open epoch, while the network is still committing; the log then shows `attempting to rejoin consensus`. | Treat it like a lagging node (next row). |
| `CATCHING UP` and the lag does not shrink between runs | Peers cannot reach the node, the disk is full or slow, or the node keeps restarting. | Check the node's P2P ports (see Ports (page 9)) in every firewall layer, check that the external multiaddrs in `node-info.yaml` carry the current public IP, check disk space, and look for restarts in the journal. |
| `eth_syncing` returns `false` while the node is behind | Expected on Telcoin Network. | Compare `eth_blockNumber` instead (see Checking by hand (page 26)). |
| `check-node.sh` warns `In the committee for epoch <E>, but the node reports Observer: it is not taking part in consensus` | The node holds a seat in this epoch's committee but is not voting, usually because it is behind or cut off from its peers. If the report adds `The node is at epoch <N>: let it catch up, and it joins once it reaches epoch <E>.`, it is still catching up. | Treat it like a lagging node (the `CATCHING UP` row). |
| `check-node.sh` reports `In the committee for epoch <E> but NOT in recent headers -- node may be running but silent` | A committee member whose headers the network does not see. | Check that the node runs, that its P2P ports are open in every firewall layer, that its external multiaddrs carry the current public IP, and that time sync is on (see Hardware per role (page 7)). |
| `check-node.sh` reports `Workers: the network requires <N> per node (...) but node-info.yaml lists <M>` | The node has fewer workers than `WorkerConfigs.numWorkers()` requires. | The scripts cannot add a worker. Email devs@telcoin.org with the `check-node.sh` output. |
| `eth_sendRawTransaction` or `eth_sendRawTransactionSync` returns error `-32000 worker is inactive` or `worker is no longer active in this epoch; submit to an active worker` | From the next release, when the network's worker count per node goes down while the node runs, the node keeps the dropped worker's RPC port open for reads but refuses transactions there. | Send transactions to the first worker's RPC, port 8545 on a default install (see Ports (page 9)). |
| `eth_sendRawTransaction` or `eth_sendRawTransactionSync` returns error `-32603 validator is inactive; retry another endpoint` | From the next release, a committee member that is catching up (`tn_nodeMode` reports `CvvInactive`) refuses new transactions until it votes again, and `/health/workers` reports it as not accepting them. | Send transactions to another node until `tn_nodeMode` reports `CvvActive`. If it stays `CvvInactive`, see the `CvvInactive` row. |

| Symptom | Likely cause | Fix |
| --- | --- | --- |
| `check-node.sh` warns `<rpc> serves chain <X> (...), not chain <Y> (...) -- skipping the network comparison` | The comparison RPC serves another chain than the node's. Today `https://rpc.telcoin.network` serves the testnet, because mainnet has not launched. | Not a node problem. Pass `--network-rpc <URL>` with an RPC for the node's chain (see Networks and endpoints (page 10)). |

## 9.2 Setup and preflight

| Symptom | Likely cause | Fix |
|---|---|---|
| Preflight prints `[WARN] <role>: below minimum` | The server is under that role's minimum. | A warning only. The check counts physical cores, so a cloud instance with hyperthreads has half as many as its vCPUs. |
| `systemd <N> is too old: the node needs 247 or later ...`, or `systemd <N> is below the mainnet floor (255, Ubuntu 24.04) ...` | The OS is older than the network needs (see Operating system (page 8)). | Move to Ubuntu 24.04. |
| `systemd <N> cannot encrypt credentials (needs 250, Ubuntu 24.04) ...` | A testnet server below systemd 250. A warning: the passphrase stays in a root-only file. Encryption needs systemd 250 or newer; Ubuntu 24.04 ships 255. | Upgrade to systemd 250 or newer, such as Ubuntu 24.04, then encrypt the passphrase with `sudo bash ~/telcoin-node-scripts/edit-config.sh --migrate-credential`. |
| `No usable TPM2 on this host: on mainnet pass --allow-no-tpm ...` | A mainnet server with no TPM the credential can use. | Give the server a TPM or vTPM, or accept the host key alone with `--allow-no-tpm` (see How setup stores the passphrase (page 15)). |
| `<tag> is not a testnet release tag ...` | A branch, a commit, `latest` or a mistyped tag. | Give a release tag such as `v0.16.0-adiri`. |
| `No release signing key in <dir> (*.asc) ...`, or `Release <tag> did not verify (see above); nothing was installed.` | The scripts carry no key for the release (one added or renewed after your scripts release), or the release's `SHA256SUMS` signature or a hash did not check out. | Run `update-scripts.sh` for the current keys and try again. If it still fails, email devs@telcoin.org with the output and do not install that release. |
| `mainnet has no pinned sha256 for genesis.yaml yet ...` | Mainnet has not launched, so the scripts have no pins for its chain configs. | Install on testnet (see Networks and endpoints (page 10)). |
| `Mainnet is not enabled in this script release (lib/common.sh TN_MAINNET_ENABLED); nothing on this host was changed.` | Mainnet has not launched, and this scripts release does not offer it. | Install on testnet, and take the release that launches mainnet with `update-scripts.sh` when it comes (see Networks and endpoints (page 10)). |
| `The host baseline was not made (see above); the node runs without it. ...` | A step of `--host-baseline` failed: no apt, the unit still holds the passphrase, or a package, write or AIDE step failed. The lines above it say which. | Fix that cause, then run `sudo bash ~/telcoin-node-scripts/setup-node.sh --host-baseline` (see Host baseline (optional) (page 22)). |
| `CVE-2026-31431 (Copy Fail) -- setup cannot continue` | The `algif_aead` module is not blocked, or is still loaded. | Apply the mitigation from https://copy.fail, unload the module or reboot, then re-run setup. |
| `Port <port>/<proto> (...) is already in use.` | Another process holds the port. | Find it with `sudo ss -lunp` (UDP) or `sudo ss -ltnp` (TCP), then stop it or choose another port. |

| Symptom | Likely cause | Fix |
| --- | --- | --- |
| `--bootstrap-peers needs telcoin-network v0.15.0-adiri or later, and ... does not have it.` (setup), or `the installed release has no --bootstrap-peers; update the node first` (`edit-config.sh`) | The node release lacks the flag. The same messages name `--enable-state-export` (`v0.13.0-adiri`) and `--state-export-keep` (`v0.15.0-adiri`). | Pick a newer release, update the node (see Update the node (page 28)), or leave the flag out. |
| `<file> is mode 600: other users cannot read it. ... Run chmod 644 <file> first.` | Setup and `edit-config.sh` take only a peers file that other users can read, because the installed copy is world-readable. | `chmod 644 <file>`, then run it again. |
| `the node rejects <file> as a --bootstrap-peers map (...)` | A key is not a BLS public key, or a value is not the `primary` and `workers` entries of that node's `node-info.yaml`. | Fix the map (see Optional node flags (page 23)). |

## 9.3 Scripts and updates

| Symptom | Likely cause | Fix |
|---|---|---|
| `lib/common.sh <version> is older than this <script>: run update-scripts.sh and try again.` | The shared library is older than the script that needs it. | `bash ~/telcoin-node-scripts/update-scripts.sh`, then run the script again. |
| `The release manifest is not signed by a Telcoin release key; nothing was changed.` | The downloaded manifest failed its signature check. | Nothing was installed. Try again later; if it keeps failing, email devs@telcoin.org. |
| `Release <tag> needs update-scripts.sh <version> or newer ...` | The release needs a newer updater than the one installed. | Run the installer command the message prints, then `update-scripts.sh` again. |
| `an update is in progress (PID <N>); try again when it has finished (edit-config.sh)`, or `Refused: an update is in progress ... (prepare-stake.sh --rotate-address)` | `update-node.sh` holds the update lock. | Wait for the update to finish, then run it again. |
| `Invalid ref "<ref>": a ref cannot start with "-".`, or `<ref> is not a testnet release tag ...` | `update-node.sh` takes release tags only. | Use a release tag such as `v0.16.0-adiri`. |
| `<ref> is older than v0.13.0, the oldest testnet release these scripts support (...)` | `update-node.sh`, or the image change in `edit-config.sh`, got a release below the network's floor. | Pick `v0.13.0-adiri` or a newer release. |
| `Image refused: on mainnet an image is named by its release tag, ...` | A mainnet Docker image given by digest alone, which `.node-meta` does not record as the image of a mainnet release the node has installed. | Give the mainnet release tag; the scripts take the digest from the signed release. |
| `Script release <tag> expired on <date>; a newer signed release is overdue.` | The scripts release is past the expiry date in its manifest. | Nothing to fix now: the release still verifies and installs. Check with `update-scripts.sh --check`; if no newer release appears soon, email devs@telcoin.org. |
| `update-node.sh` warns `... Not rolled back: the unit is active and its RPC answers, so the new release keeps running while it catches up. ...` | The node had not advanced its consensus header, or was not voting again, within `TN_UPDATE_VOTE_TIMEOUT` after the restart. | Run `check-node.sh` again in a few minutes. If it does not catch up, see Sync and node mode (page 59). |
| `... Refused: wait until the others are back, or, for a coordinated restart, run again with --allow-quorum-risk.` | Stopping this node now, with the committee members that are already down, would halt consensus. | Wait until the other members are back. Use `--allow-quorum-risk` only for a restart coordinated with the other operators (see Restarts and the epoch boundary (page 31)). |

| Symptom | Likely cause | Fix |
|---|---|---|
| `<new> is older than the running <current>: downgrades are refused.` | The release you picked is older than the one the node runs. | Pick the running release or a newer one. On testnet an interactive run with `--allow-downgrade` goes ahead, but never below `v0.16.0-adiri` on a migrated data directory, and never back to `v0.16.0-adiri` on a committee node that has run the next release (see One-way updates (page 30)). |
| `update-node.sh` ends `... Not rolled back: the previous release cannot open the migrated data dir.` | A one-way update to `v0.16.0-adiri` or later did not pass its health check, often because the first start is still migrating the consensus store. | Watch `journalctl -u telcoin -f`. A node that is still migrating comes up without help; a node that stopped says why in the journal. To go back, restore the pre-update snapshot (see One-way updates (page 30)). |
| Older release fails with `invalid version` (`invalid version (should be 0)` in the log) after a downgrade | `v0.16.0-adiri` or later has migrated the data directory, and older releases cannot read it. | Restore the data directory from the snapshot taken before the update (see One-way updates (page 30)), or run `v0.16.0-adiri` or later again. |
| `<launch file> has 2 node commands with --http, so it is not clear which one starts the node. ...` | A hand edit left a second node command in the launch file. | Remove or comment out the extra one, then run `edit-config.sh` again. |

## 9.4 Public RPC and certificates

| Symptom | Likely cause | Fix |
|---|---|---|
| `<domain> already serves the Node Manager dashboard -- the dashboard and the public RPC endpoint need DIFFERENT hostnames` | RPC and dashboard on one name. | Create `dashboard.<domain>` in DNS, then add `--move-dashboard-to dashboard.<domain>` to `rpc-enable`. |
| `rpc-check-dns` fails, or setup says `Public RPC not enabled yet` | The A record is missing, not propagated, or points at the outbound IP of a NAT host. | Fix the record, pass `--public-ip <inbound-ip>` behind NAT, then re-run `rpc-enable`. |
| The certificate is not issued, or Let's Encrypt reports a rate limit | DNS or ports 80 and 443 were wrong when Caddy asked. | Fix DNS and the ports first, then wait out the rate limit; repeated attempts extend it. |
| `port 80 is in use by 'apache2'` (or 443, or nginx) | Another web server holds the ports Caddy needs. | `sudo systemctl disable --now apache2` (or nginx), or run `sudo bash ~/telcoin-node-scripts/install-caddy.sh` and let it guide you. |
| `Caddy <version> is installed, and this script needs 2.8.0 or newer` | The installed Caddy is too old for the site `install-caddy.sh` writes. | Install the current release from Caddy's apt repository, as the message says, then run it again. |
| `This mainnet node (<address>) is a staked validator. Caddy (dashboard and public RPC) is not offered on mainnet validators ...` | `install-caddy.sh` does not serve a staked mainnet validator. | Serve the dashboard over the SSH tunnel, and public RPC from a separate RPC node. |
| `public RPC: --http.api on the node command names <modules>; ...` (or `--ws.api`) | The node serves `debug`, `trace` or `admin` methods, and Caddy passes them to anyone. | Remove them from the launch file and restart the node. |
| `public RPC: unknown (.node-meta not readable — run with sudo)` | `check-node.sh` ran without root. | Run it with `sudo`. |
| `wss://<domain>/` returns 502 | Nothing listens on the WebSocket port 8546 yet. | Wait if the node is still starting. Otherwise check `install-caddy.sh --phase=rpc-status`, and re-run `rpc-enable` if WebSocket is not listening. |

## 9.5 Service and keys

| Symptom | Likely cause | Fix |
|---|---|---|
| The node cannot decrypt its BLS key at start | The stored passphrase is not the one the keys were generated with. | Store the right one with `edit-config.sh` item `5) BLS passphrase`, `1) Change the stored passphrase`, using the copy you keep offline. It never re-encrypts the keys. |
| `... does not open the BLS key in <data dir> (keytool verify, exit N); nothing was changed. Use the passphrase the key was created with.` | `edit-config.sh` checked the passphrase with the release's `keytool verify`, and it does not open the key. | Give the passphrase the key was created with, from your offline copy. If it is lost, see Key lifecycle (page 40). |
| The service fails before the node starts after a firmware, Secure Boot or TPM change | The credential is bound to the TPM or to PCRs that changed, so systemd cannot decrypt it. | `sudo bash ~/telcoin-node-scripts/edit-config.sh --reseal`. When the old credential no longer decrypts, it asks for the passphrase, so have your offline copy ready. |
| The node will not start; the log shows `another telcoin process (pid <N>) holds the lock on this data directory` | `v0.16.0-adiri` and later lock `<data directory>/telcoin.pid`, and another node process, such as one started by hand, runs on the same data directory. | Stop the other process, then start the service. Do not delete `telcoin.pid`: the lock is released when its holder exits. |
| From the next release, the node stops at every start, and the journal shows `Cannot check database (epoch): epoch table <table> holds <n> row(s) this binary cannot decode, the first at key <key> with header epoch <N> ...`; on the first start the line begins `Cannot migrate the consensus DB from MDBX: Cannot check database (epoch)`. On `v0.16.0-adiri`, the node stops at every start with a panic `Invalid bytes!` while it opens its database | From the next release, each start checks the epoch database, which holds only the epoch in progress. It holds a row this release cannot read and every retry finds the same row, so systemd restarts the node every 10 seconds and it can end in `start-limit-hit`. On the first start, before the node has moved its database to the new format, the line names `consensus-db/epoch`; once it has moved, the line names `consensus-db/tndb/epoch`. `v0.16.0-adiri` runs no such check: a damaged row in its epoch database makes the node panic as it opens the database, with the same restarts. | When the line begins `Cannot migrate the consensus DB from MDBX` and names `consensus-db/epoch`, stop the node (`sudo systemctl stop telcoin`) and move that directory out of the data directory, for example `sudo mv /var/lib/telcoin/consensus-db/epoch /var/lib/telcoin.epoch-<N>`, and keep `consensus-db/epochs` and `db` in place. That directory also held the node's guards against voting or proposing twice in epoch `<N>`, so start the node again only once the network has closed that epoch: compare `<N>` with the current epoch (see The epoch section (page 25)). When the line names `consensus-db/tndb/epoch`, do not remove or move it, even though the journal line suggests it: the next start refuses an epoch database that is missing or empty beside the marker `consensus-db/tndb/mdbx-migrated`, so wait for an answer from devs@telcoin.org. On `v0.16.0-adiri`, send the journal lines with the `Invalid bytes!` panic and wait for an answer. A node with state export on can instead be restored into a fresh data directory with `db load-state`. |
| `systemctl status telcoin` shows `start-limit-hit` | Five failed starts within 60 seconds. | Fix the cause from the journal (for `Cannot check database (epoch)`, see the row above), then `sudo systemctl reset-failed telcoin && sudo systemctl start telcoin`. |

| Symptom | Likely cause | Fix |
| --- | --- | --- |
| After a restart near an epoch boundary, the log shows `previous epoch record missing on restart (killed at its boundary); re-deriving it locally`, `epoch record present but the consensus db tables still hold rows of an older epoch (...); clearing them` or `ignoring parent certificates from another epoch` | The node was stopped while the epoch changed, and it repairs its epoch records as it starts. | Normal, no action needed. For planned restarts, keep to the epoch wait (see Restarts and the epoch boundary (page 31)). |
| An `ERROR` line ending `; ending the epoch (the node stops if its stores cannot write): <error>`, and the node exits; on `v0.16.0-adiri` the line for a vote ends `; initiating node shutdown: <error>`, and the node keeps running | The node could not write a vote or certificate durably to its epoch database (see Where things land (page 17)) and refused to send it rather than risk signing twice. From the next release it then stops itself, because that store refuses every later write until a restart: it exits with status 1, and systemd starts it again after 10 seconds. On `v0.16.0-adiri`, after a failed vote write, it stops only its consensus tasks and keeps running, but it cannot vote again until it is restarted. | Check the data drive: free space with `df -h /var/lib/telcoin` and I/O errors with `sudo dmesg`, and fix the cause. A disk that still fails makes every restart fail too, and systemd can give up with `start-limit-hit`; then run `sudo systemctl reset-failed telcoin && sudo systemctl start telcoin`. On `v0.16.0-adiri`, restart the node at once with `sudo systemctl restart telcoin` once the cause is fixed; the epoch wait protects nothing while the node cannot vote. If the line comes back, email devs@telcoin.org with it. |

## 9.6 Staking and exit

| Symptom | Likely cause | Fix |
|---|---|---|
| `prepare-stake.sh` ends `Not ready: 0x... is not whitelisted.` | The address holds no ConsensusNFT yet. | Send the address to the Association (devs@telcoin.org) and run the script again once the NFT is minted. |
| `prepare-stake.sh` ends `Not ready: 0x... is short of TEL.` | The balance is below the stake plus gas. | Send at least the amount the line above it names, then run it again. |
| `prepare-stake.sh` stops with `<rpc> serves chain <X> (...), not chain <Y> (...)` | The network RPC serves another chain than the node's. | Pass `--network-rpc <URL>` with an RPC for the node's chain (see Networks and endpoints (page 10)). |
| `stake` reverts, or `prepare-stake.sh` ends `Not ready: stake() would revert with <Error>` | The registry would refuse the stake. | Look the error up in If prepare-stake.sh predicts a revert (page 48). |
| `activate` reverts with `InvalidStatus` | The validator is not in the status the call needs. | Check the status (see Watch your status (page 49)). |
| `check-node.sh` warns `VALIDATOR_ADDRESS in .node-meta (...) is not the node's execution address (...)` | `.node-meta` names another address than `node-info.yaml`, for example after a hand edit. The stake status is read for the `.node-meta` address. | Correct `VALIDATOR_ADDRESS` in `/etc/telcoin/.node-meta`, or pass `--address`. |
| `No BLS key passphrase. Set TN_BLS_PASSPHRASE ...` | `--rotate-address` found no passphrase source. | Use one of the sources in Change the execution address before you stake (page 49). |
| `Refused: 0x... has staked (...)` | The address has staked, so its BLS key stays registered to it. | Stake from that address; it cannot change now. |
| `prepare-stake.sh --rotate-address` exits 4 with `keytool generate pop failed (exit <N>): ... node-info.yaml is back as it was (from <backup>).` | Usually a wrong passphrase. | Run it again with the passphrase the keys were generated with. |
| `beginExit` reverts | The validator is not Active yet, or too few validators would remain. | Wait, then retry. |
| `unstake` reverts with `IneligibleUnstake` | The validator is neither Staked nor one epoch past Exited. | Wait for Exited plus one epoch. |

# 10 Getting help

Email devs@telcoin.org for everything, including validator onboarding, approval and hardware.

Include:

- The full output of `sudo bash ~/telcoin-node-scripts/check-node.sh`.
- For staking questions, the output of `sudo bash ~/telcoin-node-scripts/prepare-stake.sh`.
- The last 200 log lines: `journalctl -u telcoin -n 200 --no-pager`.
- The node release (`RELEASE_TAG` in `/etc/telcoin/.node-meta`, or the binary's `--version` on an `existing` install) and the output of `bash ~/telcoin-node-scripts/update-scripts.sh --check`.
- The network (testnet or devnet) and your install method.

> **WARNING**
>
> Never send the contents of `node-keys/` or your BLS passphrase, to support or to anyone else.